RV College of Engineering®

Artificial Intelligence and Machine learning

IS353IA

Unit 2

Mekhala V Purohit

Assistant Professor

Department of Computer Science and Engineering

RVCE

All material presented in this PowerPoint is credited to the respective authors of standard textbooks, reference presentations shared by senior faculty at **RVCE**, and authentic internet sources.

Note to Students:

This presentation is meant **only for classroom reference and conceptual understanding**.

For exam and detailed preparation, please refer to the **textbook extracts and materials shared in Google Classroom**, as the content here is a **summary version** and not exhaustive.

Go, change the world®

- **Informed Search Methods**
  - → Heuristic Functions
  - → A* Search
- **Beyond Classical Search**
  - → Local Search & Optimization
  - → Hill-climbing Search
  - → Simulated Annealing
  - → Local-Beam Search
  - → Genetic Algorithms
- **Game Playing**
  - → Introduction
  - → Game as a Search problem
  - → Perfect Decisions in Two-Persons
  - → Games Imperfect Decisions
  - → Alpha-Beta Pruning

Informed Search Methods

Introduction

- **Informed Search Strategy**
  - One that uses problem-specific knowledge beyond the definition of the problem itself
  - This finds the solutions more efficiently than uninformed strategy
- **For example**
  - The search algorithm comprise of the knowledge like how far the goal is, path cost, how to reach to goal node, etc.
  - This knowledge help agents to explore less to the search space and find more efficiently the goal node

RV College of Engineering®

Heuristics Function

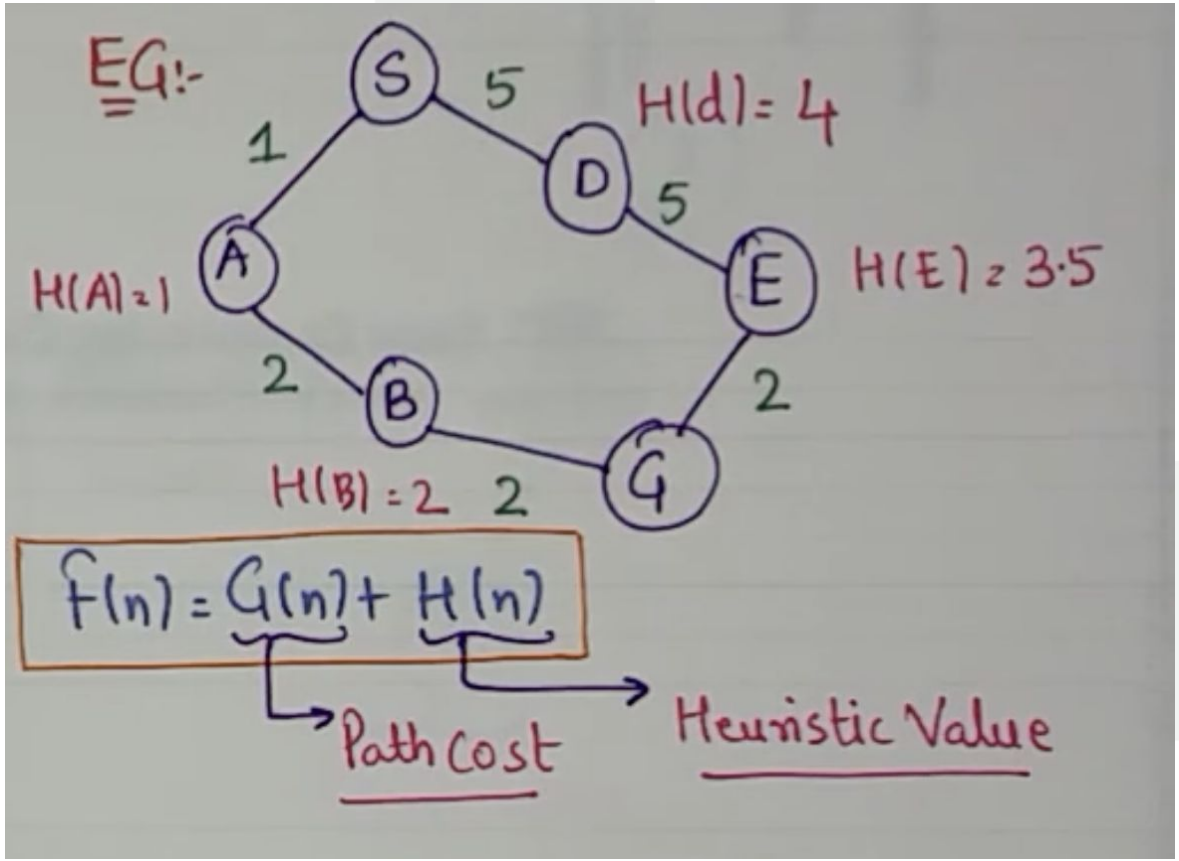
- Heuristic is a function which is used in Informed Search, and it finds the most promising path.
- It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal
- The heuristic method, might not always give the best solution, but it guaranteed to find a good solution in reasonable time
- Heuristic function estimates how close a state is to the goal
- It is represented by h(n), and it calculates the cost of an optimal path between the pair of states
- The value of the heuristic function is always positive

f(n) = h(n)

Here, f(n) is heuristic cost, and h(n) is the estimated cost. Hence heuristic cost should be less than or equal to the estimated cost.

RV College of Engineering®

Heuristics Function

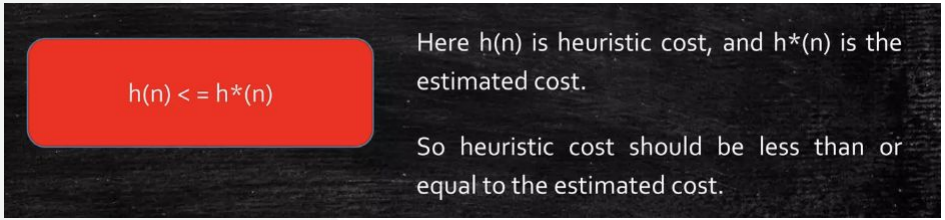


RV College of Engineering®

Heuristics Function

1) Admissible heuristic function – A heuristic function is admissible if it underestimates the cost of reaching the goal.
"**Underestimates** or at most equals the real cost"

Meaning:
- The heuristic must **never overestimate** the real cost.
- It can be smaller or exactly equal, but **not bigger**.



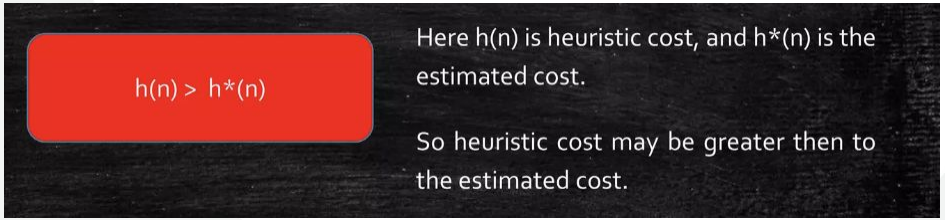
A heuristic function **h(n)** gives an estimate of how far the current node **n** is from the goal.
There is also a "perfect" heuristic called **h*(n)**:
- **h*(n)** = the **true**, actual minimum cost from node **n** to the goal
(we don't know this in real problems, but it is used for theory)

Why does admissible matter?
Because, using an admissible heuristic is **guaranteed to find the optimal path**.

RV College of Engineering®

Non Admissible heuristic function

2) Non Admissible heuristic function – may overestimate the cost of reaching the goal.
"May **overestimate** the real cost"

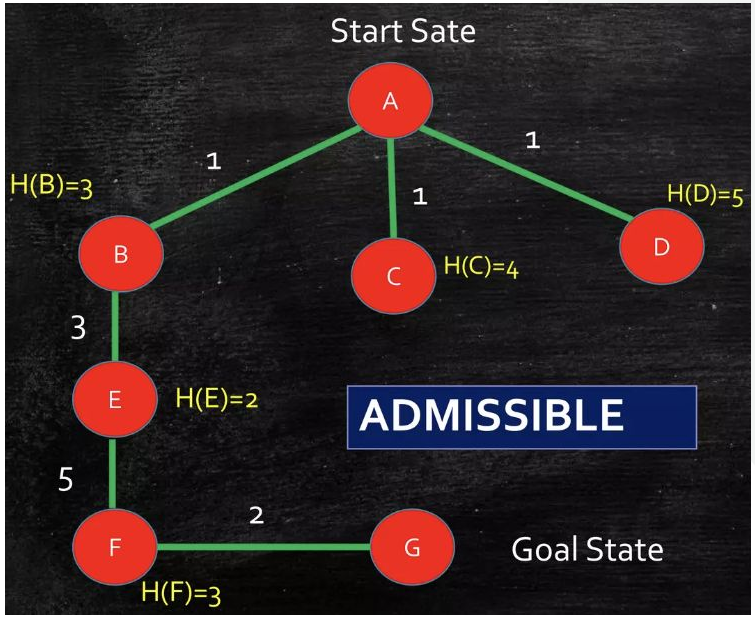


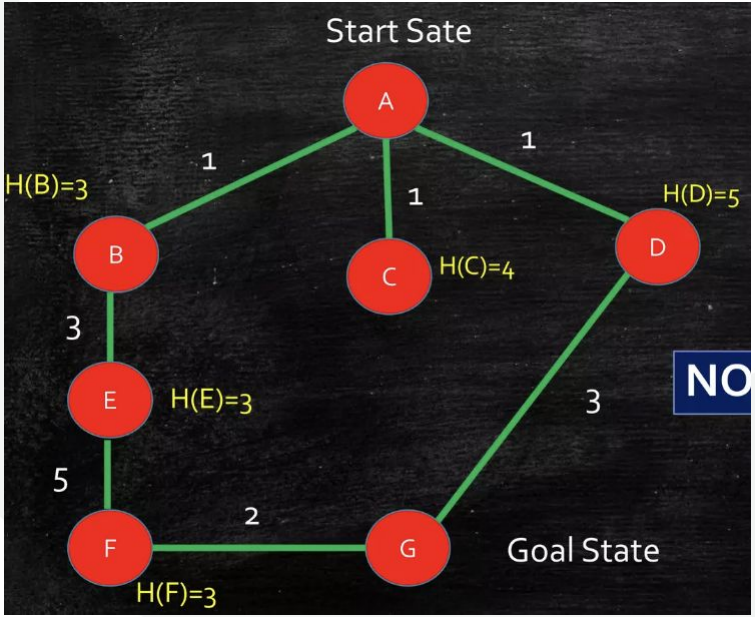
Meaning:

- The heuristic might give a value **higher** than the actual cost.
- It is not safe for optimality.
- A* may take shortcuts or wrong paths → might not find the best path.

RV College of Engineering®

Heuristics Function





RV College of Engineering®

From Goal G:

- **h*(G) = 0**

From F → G

F → G = 2

- **h*(F) = 2**

From E → F → G

E → F → G = 5 + 2 = 7

- **h*(E) = 7**

From B → E → F → G

B → E → F → G = 3 + 5 + 2 = 10

- **h*(B) = 10**

From D → G

D → G = 3

- **h*(D) = 3**

From C (two ways):

C → A → D → G = 1 + 1 + 3 = 5

Best: **5**

- **h*(C) = 5**

From A (best path):

A → D → G = 1 + 3 = 4

- **h*(A) = 4**

Node	Actual Cost to Goal h*(n)
A	4
B	10
C	5
D	3
E	7
F	2
G	0

RV College of Engineering®

Left graph

Node	h(n)	h*(n)	Admissible?
B	3	10	✔ 3 ≤ 10
C	4	5	✔ 4 ≤ 5
D	5	3	❗ 5 > 3 → overestimates
E	2	7	✔
F	3	2	❗ 3 > 2 → overestimates
G	0	0	✔

RV College of Engineering®

The 8 Puzzle Problem

Board: a 3×3 grid with **8 numbered tiles** and **one blank**.

- An action = slide a tile horizontally or vertically into the blank.
- Goal: reach the **goal configuration** from a given **start configuration** using the fewest moves.
- The search space size (reachable states) = 9!/2=181,440 states (classic fact).

RV College of Engineering®

The two heuristics shown on the slide

h1 = number of misplaced tiles

- Count of tiles that are not in their goal position.
- Example: if 3 tiles are wrong, h1 = 3.
- Admissible? Yes — each misplaced tile needs at least one move, so h1 never overestimates the true minimal number of moves.

h2 = sum of Manhattan distances (city-block distance)

- For each tile, compute the distance from its current cell to its goal cell as:

$$\text{Manhattan} = |r_{cur} - r_{goal}| + |c_{cur} - c_{goal}|$$

- h2 = sum of those distances for tiles 1……8.
- Admissible? Yes — moving a tile one step changes its Manhattan distance by at most 1, so the sum of Manhattan distances is a lower bound on the number of moves required.

RV College of Engineering®

Heuristics Function (Contd.)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

- The 8-puzzle was one of the earliest heuristic search problems
- The objective of the puzzle is to slide the tiles horizontally or vertically into the empty space until the configuration matches the goal configuration
- Here, there are two commonly used candidates:
  - h1 = the number of misplaced tiles, all of the eight tiles are out of position, so the start state would have h1 = 8
  - h2 = the sum of the distances of the tiles from their goal positions
    - The distance we will count is the sum of the horizontal and vertical distances. This is sometimes called the city block distance or Manhattan distance
    - Tiles 1 to 8 in the start state give a Manhattan distance of h2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18

RV College of Engineering®

Greedy Best First Search

Greedy Best-First Search is a search strategy that:

- Always expands the node that appears closest to the goal

(based ONLY on the heuristic value h(n))

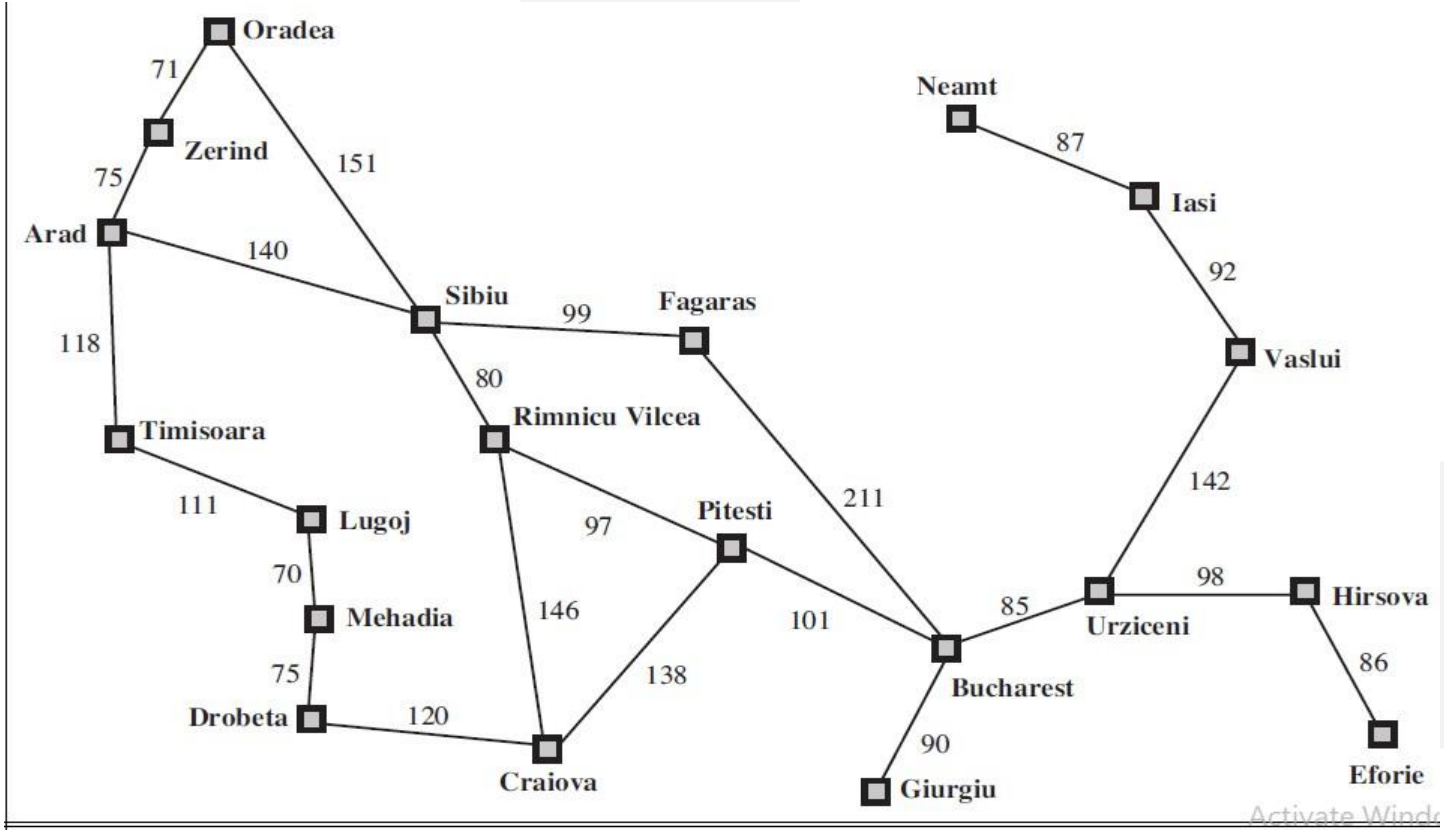
It does not consider:

- the cost so far (g(n)),
- the total cost (f(n)),
- or optimality.

It is called “greedy” because it greedily chooses the best-looking next node.

RV College of Engineering®

Greedy Best First Search



RV College of Engineering®

Greedy Best First Search

- Greedy best first search tries to expand the node that is closest to the goal- on the grounds that it is likely to lead to a solution quickly.
- Route finding problems in Romania
- Heuristic- Straight line distance hSLD
- If the goal is Bucharest, we need to know SLD to Bucharest
- Notice that the values of hSLD cannot be computed from the problem description itself. Moreover, it takes a certain amount of experience to know that hSLD is correlated with actual road distances and is, therefore, a useful heuristic.

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

Figure 3.22 Values of h_{SLD}—straight-line distances to Bucharest.

Greedy Best First Search

- Greedy best-first tree search is incomplete even in a finite state space, much like depth-first search.
- Consider the problem of getting from Iasi to Fagaras. The heuristic suggests that Neamt be expanded first because it is closest to Fagaras, but it is a dead end. The solution is to go first to Vaslui—a step that is actually farther from the goal according to the heuristic—and then to continue to Urziceni, Bucharest, and Fagaras.
- The algorithm will never find this solution, however, because expanding Neamt puts Iasi back into the frontier, Iasi is closer to Fagaras than Vaslui is, and so Iasi will be expanded again, leading to an **infinite loop**. (The graph search version *is complete in finite spaces, but not in infinite ones.)*
- *The* worst-case time and space complexity for the tree version is O(bm), where m is the maximum depth of the search space. With a good heuristic function, however, the complexity can be reduced substantially.
- The amount of the reduction depends on the particular problem and on the quality of the heuristic.

Heuristics Function (Contd.)

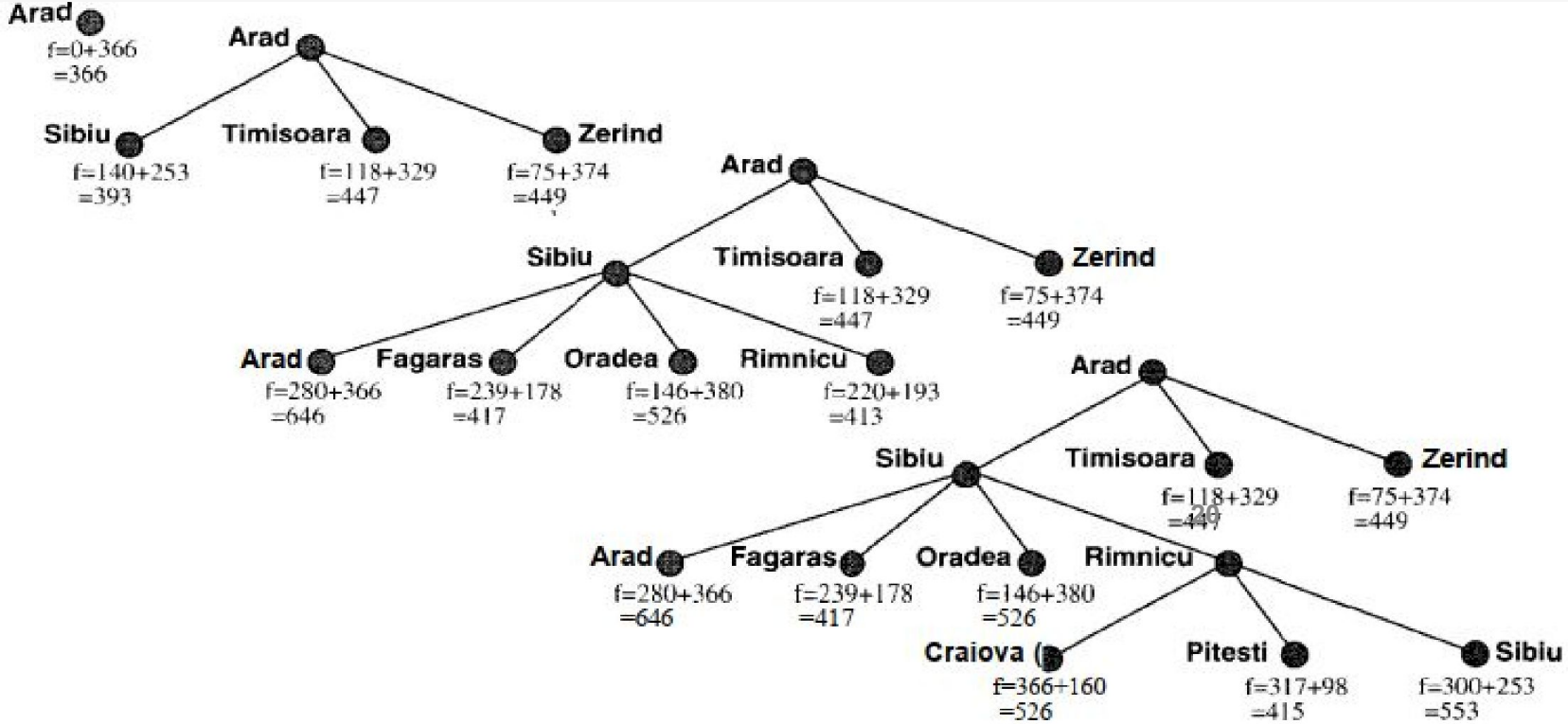
→ Pure Heuristic Search(GBFS)

- Pure heuristic search is the simplest form of heuristic search algorithms
- It expands nodes based on their heuristic value h(n)
- It maintains two lists, **OPEN** and **CLOSED** list
- In the **CLOSED** list, it places those nodes which have already expanded
- **In OPEN** list, it places nodes which have yet not been expanded
- On each iteration, each node 'n' with the lowest heuristic value is expanded and generates all its successors and 'n' is placed to the closed list
- The algorithm continues until a goal state is found

Heuristics Function (Contd.)

Go, change the world

→ A* Search Algorithm



Heuristics Function (Contd.)

RV College of Engineering®

→ A* Search Algorithm

- A* search is the most commonly known form of best-first search
- It uses heuristic function h(n), and cost to reach the node 'n' from the start state g(n)
- A* search algorithm finds the shortest path through the search space using the heuristic function
- This search algorithm expands less search tree and provides optimal result faster
- In A* search algorithm, we use search heuristic as well as the cost to reach the node. Hence we can combine both costs as following, and this sum is called as a fitness number

Heuristics Function (Contd.)

→ A* Search Algorithm

f(x) = g(x)+h(x)
g(x) - backward cost
h(x) - forward cost

Step1: Place the starting node in the OPEN list.

Step 2: Check if the OPEN list is empty or not, if the list is empty then return failure and stops.

Step 3: Select the node from the OPEN list which has the smallest value of evaluation function (g+h), if node n is goal node then return success and stop, otherwise

Step 4: Expand node n and generate all of its successors, and put n into the closed list. For each successor n', check whether n' is already in the OPEN or CLOSED list, if not then compute evaluation function for 'n' and place into Open list.

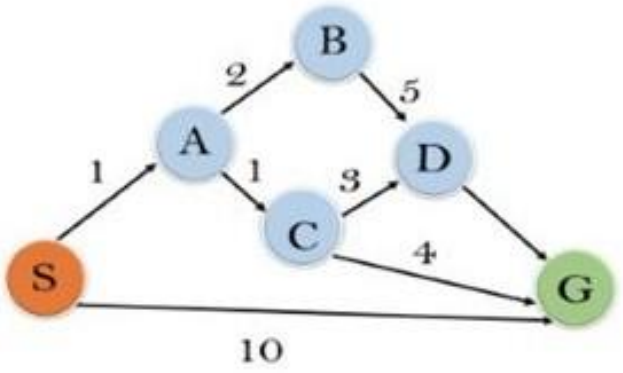
Step 5: Else if node n' is already in OPEN and CLOSED, then it should be attached to the back pointer which reflects the lowest g(n') value.

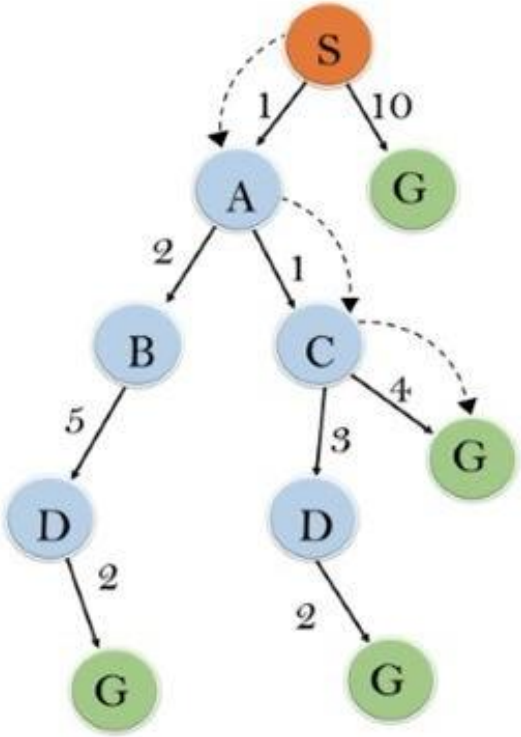
Step 6: Return to **Step 2**

Heuristics Function (Contd.)

RV College of Engineering®

→ **A* Search Algorithm**





Node	H(n)
S	5
A	3
B	4
C	2
D	6
G	0

RV College of Engineering®

Memory Bounded Search

⟹ Iterative deepening A* search (IDA*)

- Iterative deepening A* (IDA*) is a graph traversal and path search algorithm
- The algorithm can find the shortest path between a designated start node and any member of a set of goal nodes in a weighted graph
- It is a variant of iterative deepening depth-first search that borrows the idea to use a heuristic function to evaluate the remaining cost to get to the goal from the A* search

```
function IDA*(problem) returns a solution sequence
  inputs: problem, a problem
  static: f-limit, the current f-COST limit
          mot, a node

  root ← MAKE-NODE(INITIAL-STATE[problem])
  f-limit ← f-COST(root)
  loop do
     solution, f-limit — DFS-CONTOUR(root,f-limit)
     if solution is non-null then return solution
     itf-limit = ∞ then return failure; end
```

```
function DFS-CONTOUR(node,f-limit) returns a solution sequence and a new f-COST limit
  inputs: node, a node
          f-limit, the current f-COST limit
  static: next-f, the f-COST limit for the next contour, initially ∝

  if f-COST[node] > f-limit then return null, f-COST[node]
  if GOAL-TEST[problem](STATE[node]) then return node, f-limit
  for each node s in SUCCESSORS(node) do
     solution, new-f — DFS-CONTOUR(s,f-limit)
     if solution is non-null then return solution, f-limit
     next-f ← MIN(next-f, new-f); end
  return null, next-f
```

A* stores EVERY generated node in OPEN and CLOSED lists.

For large graphs:

- Memory consumption becomes huge
- A* may fail due to memory overflow

IDA* solves this problem by using:

- Depth-first search (DFS-like) exploration
- Limited by increasing thresholds of f(n)

No huge OPEN/CLOSED lists

A* expands nodes in order of increasing f(n).

IDA* copies the same idea, but:

- It uses DFS instead of storing nodes in a priority queue
  It sets an f-limit and only explores nodes with

$$f(n) \leq f\text{-limit}$$

- If no solution is found, it increases the limit and tries again.

This is why it's called Iterative Deepening, like Iterative Deepening DFS, but guided by f-values instead of depth.

Step 1:

- Compute f(start) = g(start) + h(start).
  This = initial f-limit.

Step 2:

- Run DFS, but cut off paths where:
- f(n)>f-limit

Step 3:

- If you reach the goal, stop — solution found.

Step 4:

- If the search stops because all nodes exceeded the limit:
- Increase the f-limit to the smallest f(n) that exceeded the limit, Run DFS again

Step 5:

- Repeat until the goal is found.

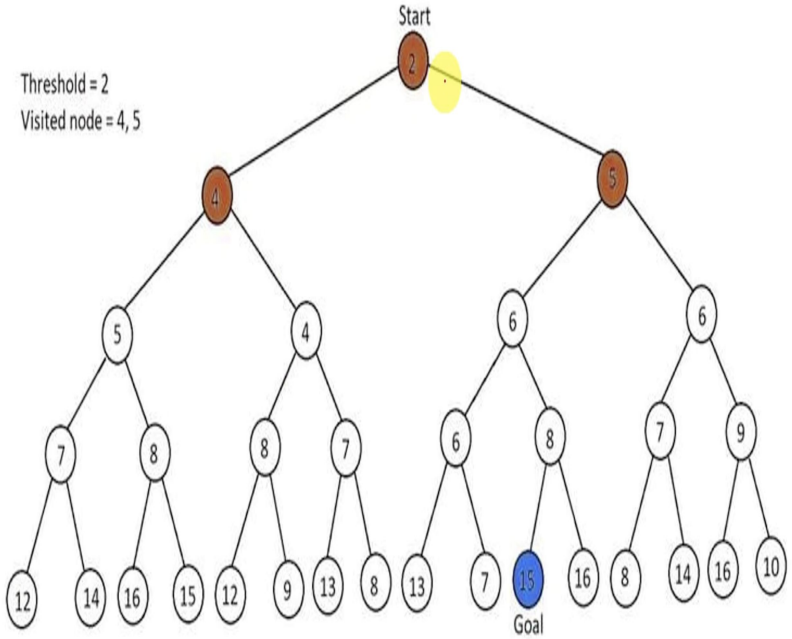
Iterative Improvement Algorithms (Contd.)

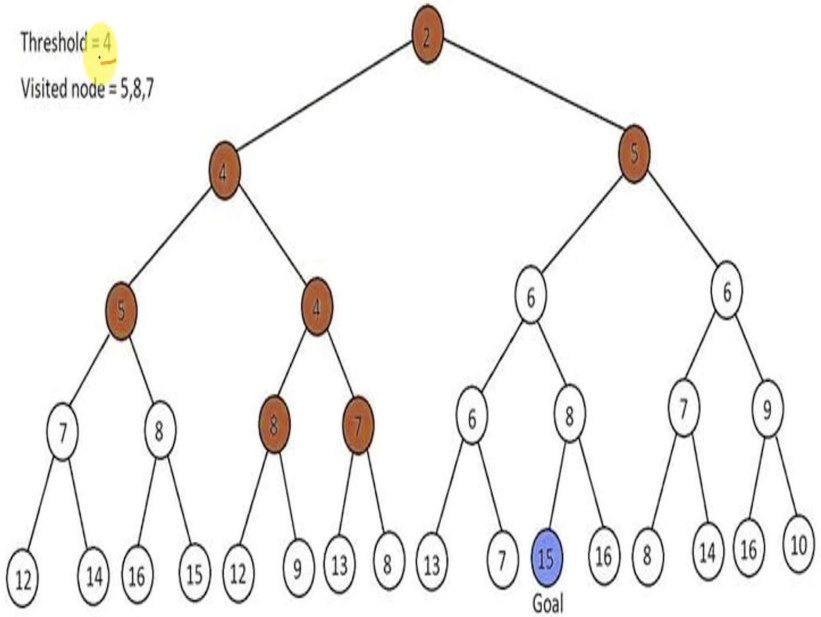
- Unlike A*, IDA* doesn't utilize dynamic programming and therefore often ends up exploring the same nodes many times
- It does everything that the A* does, it has the optimal characteristics of A* to find the shortest path but it uses less memory than A*.

A* uses exponential memory: O(b to the power d)
IDA* uses linear memory: O(d)

RV College of® Engineering®

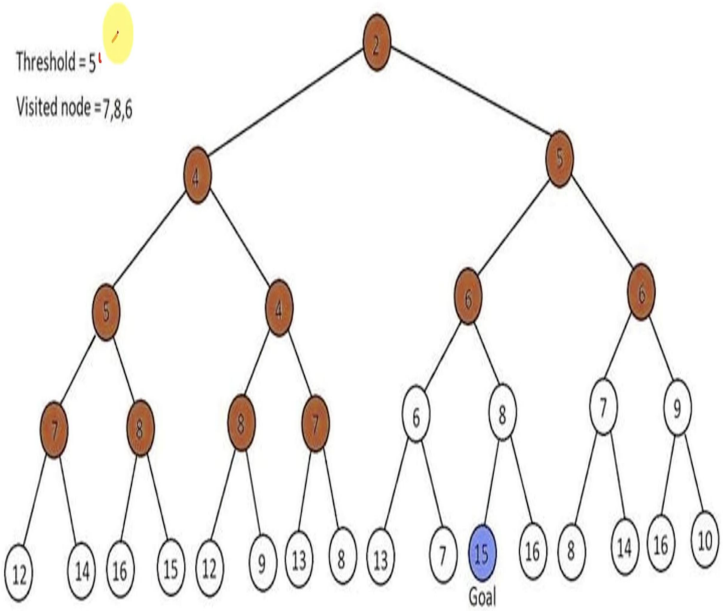
Iterative Improvement Algorithms (Contd.)

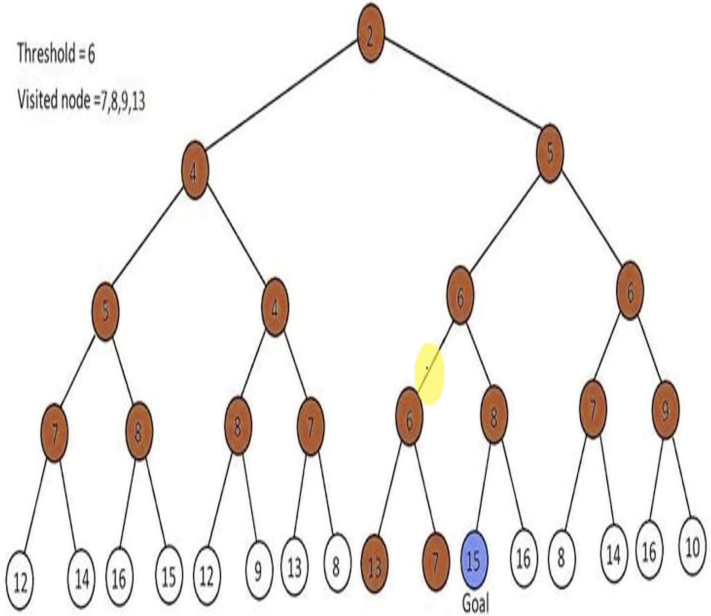




RV College of Engineering®

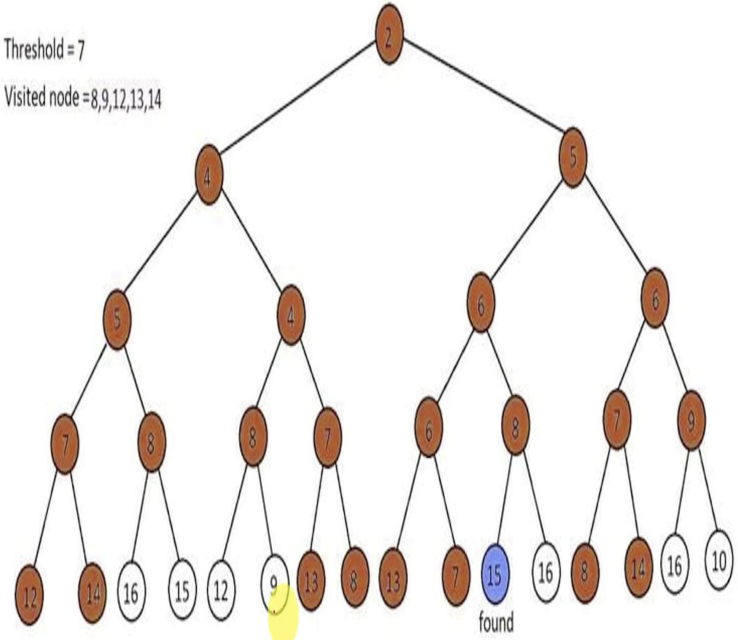
IDA*

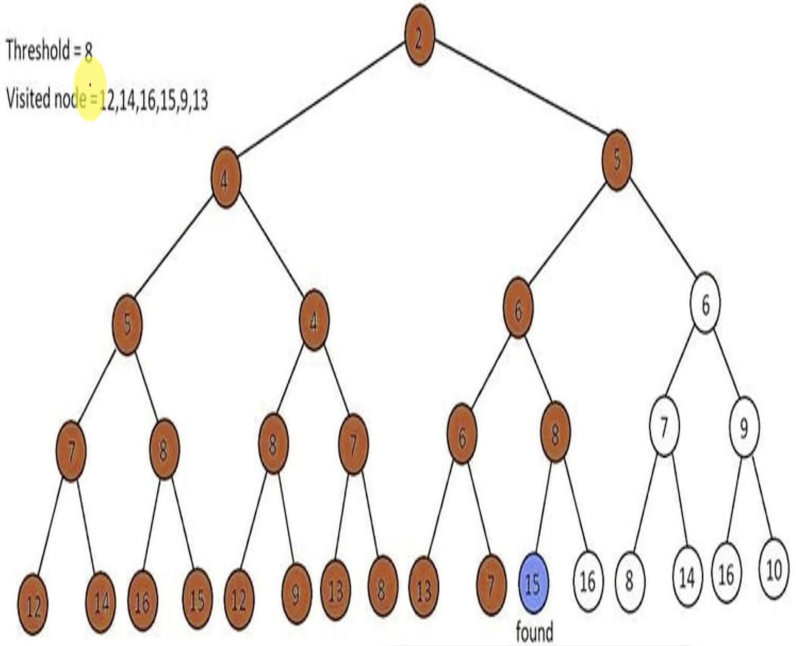




RV College of Engineering®

IDA*





Iterative Improvement Algorithms (Contd.)

Simplified Memory-Bounded A*

- SMA* is a shortest path algorithm that is based on the A* algorithm.
- The difference between SMA* and A* is that SMA* uses a bounded memory, while the A* algorithm might need exponential memory.
- SMA* has the following properties:
  - It will utilize whatever memory is made available to it.
  - It avoids repeated states as far as its memory allows.
  - It is complete if the available memory is sufficient to store the shallowest solution path.
  - It is optimal if enough memory is available to store the shallowest optimal solution path.
  - Otherwise, it returns the best solution that can be reached with the available memory.
  - When enough memory is available for the entire search tree, the search is optimally efficient

Iterative Improvement Algorithms (Contd.)

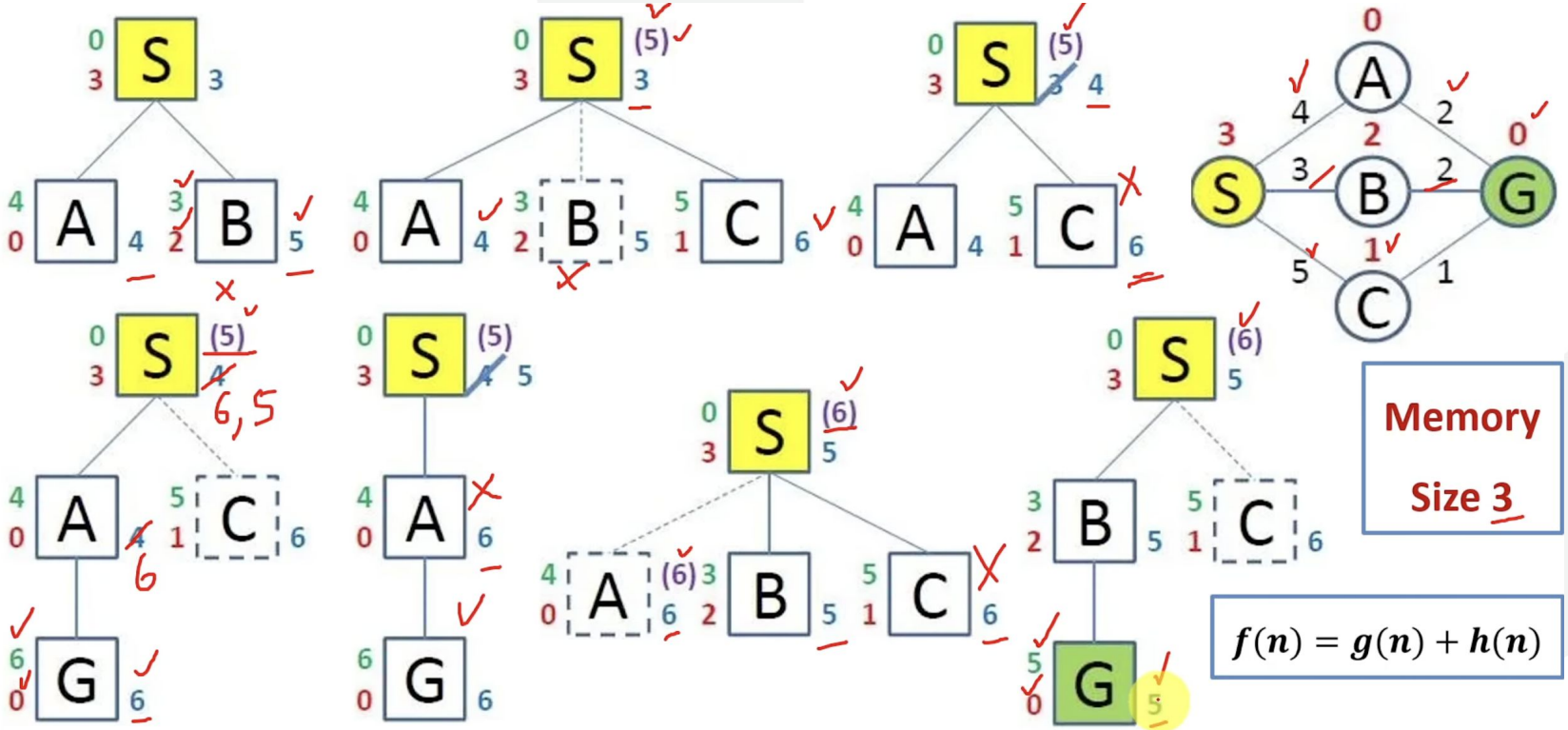
Simplified Memory-Bounded A*

- SMA*, just like A* evaluates nodes by combining g(n), the cost to reach the node, and h(n), the cost to get from he node to the goal: **f(n) = g(n) + h(n)**
- Since g(n) → is the path cost from the start node to node n

  h(n) → is the estimated cost of the cheapest path from n to the goal

  f(n) → estimated cost of the cheapest solution through n
- The lower the f value is, the higher priority the node will have
- The difference from A* is that the f value of the parent node will be updated to reflect changes to this estimate when its children are expanded
- A fully expanded node will have an **'f'** value at least as high as that of its successors
- In addition, the node stores the f value of the best forgotten successor (or best forgotten child). This value is restored if the forgotten successor is revealed to be the most promising successor.

Iterative Improvement Algorithms (Contd.)



Iterative Improvement Algorithms

Simplified Memory-Bounded A*

- In many optimization problems, path is irrelevant
- In such cases, can use iterative improvement algorithms; keep a single “current” state, try to improve it
- Iterative improvement algorithms divide into two major classes
  - Hill-climbing (gradient descent)
  - Simulated annealing
- Hill-Climbing (Gradient Descent)
  - A hill-climbing algorithm is an Artificial Intelligence (AI) algorithm that increases in value continuously until it achieves a peak solution
- Simulated Annealing
  - Simulated Annealing (SA) mimics the Physical Annealing process but is used for optimizing parameters in a model

Beyond Classical Search

Applications of Search Problem

- integrated-circuit design
- factory floor layout
- job shop scheduling
- automatic programming
- telecommunications network optimization
- crop planning
- portfolio management

Sometimes, In these applications especially we care only about the final state, not the path to get there.

That is where your local search can be used.

RV College of Engineering®

Local Search

1. What are Local Search Algorithms?

- Local search algorithms start from an initial (start) state.
- They move only to neighboring states (small variations of the current state).
- They do not store the path taken to reach a state.
- They do not keep a record of all visited states.

2. How They Work (Core Idea)

- At any time, the algorithm:
  - Looks at the current state
  - Evaluates its neighbors
  - Moves to a better neighboring state based on some evaluation (heuristic/cost)
- The search continues until a goal or good enough solution is found.

Local Search

3. Why They Are Called "Not Systematic"

- They do not explore the entire search space methodically.
- Some regions of the state space may never be visited.
- Because of this:
  - A solution **might exist**, but the algorithm **may never reach it**.
  - It can get **stuck in local optima, plateaus, or ridges**.

Examples of Local Search Algorithms

- Hill Climbing
- Simulated Annealing
- Local Beam Search
- Genetic Algorithms

Local search and optimization

"Pure optimization" problems

- All states have an objective function
- Goal is to find state with max (or min) objective value
- Does not quite fit into path-cost/goal-state formulation

In pure optimization problems:

- **Every possible state has a value** given by an **objective function**.
- There is **no single goal state**.

The task is **not to reach a destination**, but to **find the best state**.

Objective Function

•An **objective function** assigns a numerical value to each state.

•The algorithm aims to:

- **Maximize** the value (e.g., profit, accuracy, fitness), or
- **Minimize** the value (e.g., cost, error, energy).

Goal of the Problem

•Goal = **Find the state with the maximum or minimum objective value**

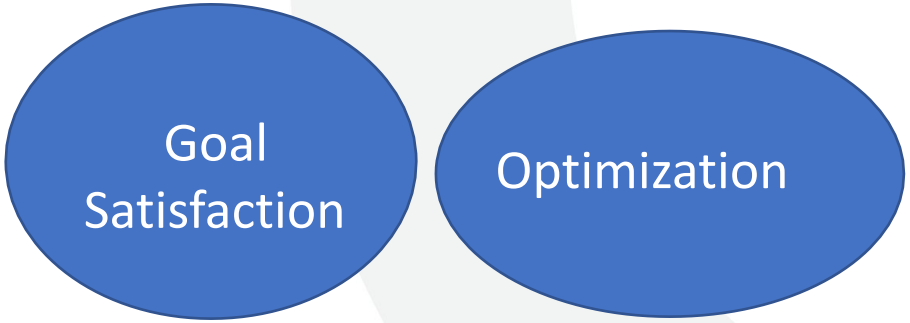
•The "best" state is called the **optimal solution**.

RV College of Engineering®

Local Classical Algos. and Optimization Problems

Goal Satisfaction

- **Objective:** Reach *any* state that satisfies all given conditions.
- **Focus:**"Is the goal reached?" → **Yes / No**
- **Nature of solution:** Any valid solution is acceptable.



Optimization

- **Objective:** Find the *best* state according to an **objective function**.
- **Focus:**Maximize or minimize a numerical value.
- **Nature of solution:**Compare solutions → choose the **best one**.

You can go back and forth between the two problems
Typically in the same complexity class

Local Classical Algos. and Optimization Problems

- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution
  - Local search: widely used for very big problems
  - Returns good but not optimal solutions in general
- The state space consists of "complete" configurations
  - For example, every permutation of the set of cities is a configuration for the traveling salesperson problem
- The goal is to find a “close to optimal” configuration satisfying constraints
  - Examples: n-Queens, VLSI layout, exam time table

- Local search algorithms
  - Keep a single "current" state, or small set of states
  - Iteratively try to improve it / them
    - Very memory efficient since only a few states are stored

Local Classical Algos. and Optimization Problems

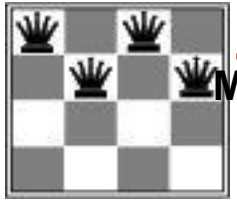
Example: 4-queens

Goal: Put 4 queens on an 4 × 4 board with no two queens on the same row, column, or diagonal

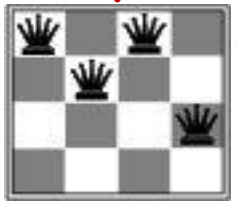
State space: All configurations with the queens in distinct columns

State transition: Move a queen from its present place to some other square in the same column

Local Search: Start with a configuration and repeatedly use the moves to reach the goal



Move queen in Column 4



Move queen in Column 2

The last configuration has fewer conflicts than the first, but is still not a solution

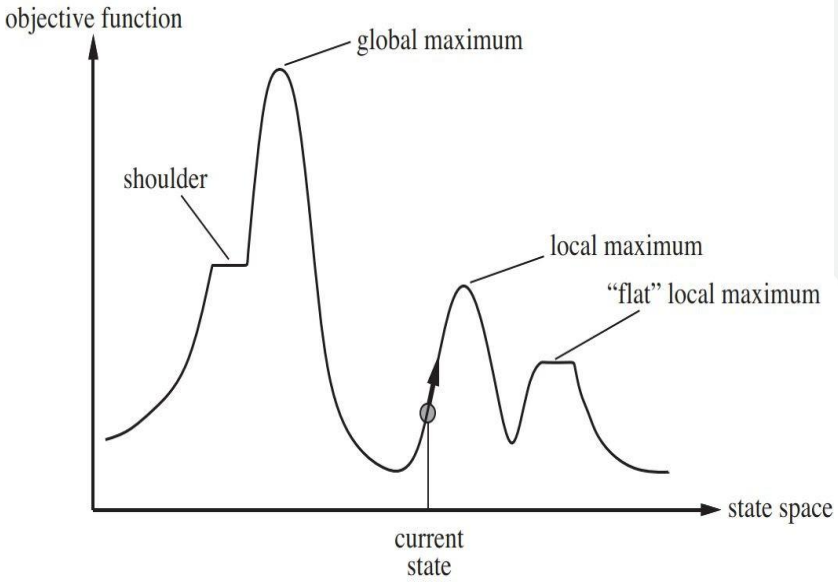
Hill Climbing

- A hill-climbing algorithm is a local search algorithm that moves continuously upward (increasing) until the best solution is attained
- This algorithm comes to an end when the peak is reached
- This algorithm has a node that comprises two parts: state and value. It begins with a non-optimal state (the hill's base) and upgrades this state until a certain precondition is met
- The heuristic function is used as the basis for this precondition. The process of continuous improvement of the current stateof iteration can be termed as climbing. This explains why the algorithm is termed as a hill-climbing algorithm
- A hill-climbing algorithm has four main features
  - It employs a **greedy approach**
  - **No Backtracking:** It does not look at the previous states
  - **Feedback mechanism:** The algorithm has a feedback mechanism that helps it decide on the direction of movement
  - **Incremental change:** The algorithm improves the current solution by incremental changes

RV College of Engineering®

Hill Climbing

- The following diagram shows a simple state-space diagram

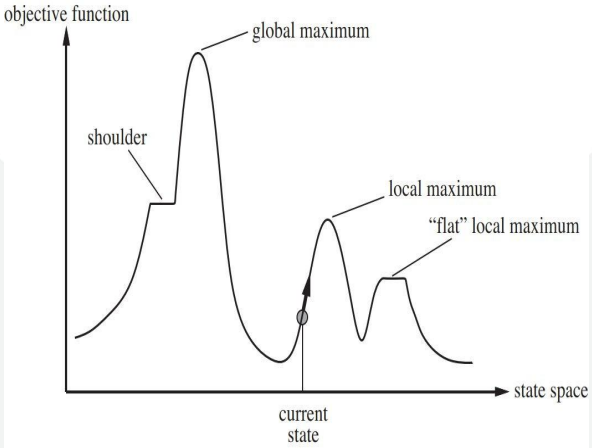


- **Local maximum:** A local maximum is a solution that surpasses other neighboring solutions or states but is not the best possible solution
- **Current state:** This is the existing or present state
- **Flat local maximum:** This is a flat region where the neighboring solutions attain the same value
- **Shoulder:** This is a plateau whose edge is stretching upwards
- **Global maximum:** This is the best possible solution achieved by the algorithm

RV College of Engineering®

1. Current State

- The current state is the solution the algorithm is at right now.
- All decisions (next move) are made by:
  - Comparing this state with its neighboring states.
- It keeps changing as the search progresses.



2. Local Maximum

- A local maximum is a state that:
  - Is better than all its neighboring states
  - But is not the best overall solution

The algorithm may stop here, thinking no improvement is possible.

Better than nearby, but not the best globally.

RV College of Engineering®

3. Flat Local Maximum (Plateau)

A flat local maximum is a flat region in the search space where:

- Many neighboring states have exactly the same value

The algorithm:

- Cannot decide which direction is better
- May wander randomly or get stuck

No improvement, no decline — just flat.

4. Shoulder

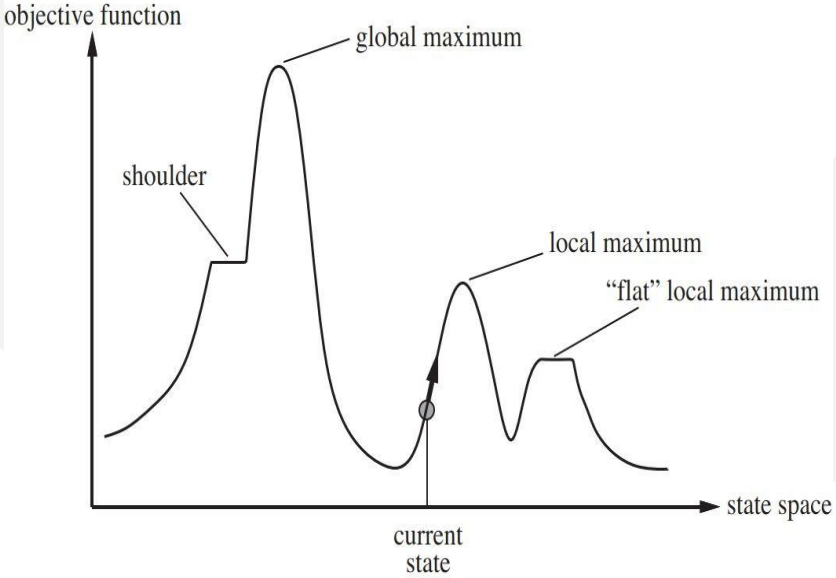
A shoulder is a special kind of plateau where:

- Most neighbors have the same value
- But one edge leads upward to a better solution

If the algorithm moves correctly, it can:

- Escape the plateau
- Reach a higher value

Looks flat, but has a hidden upward path.



RV College of Engineering®

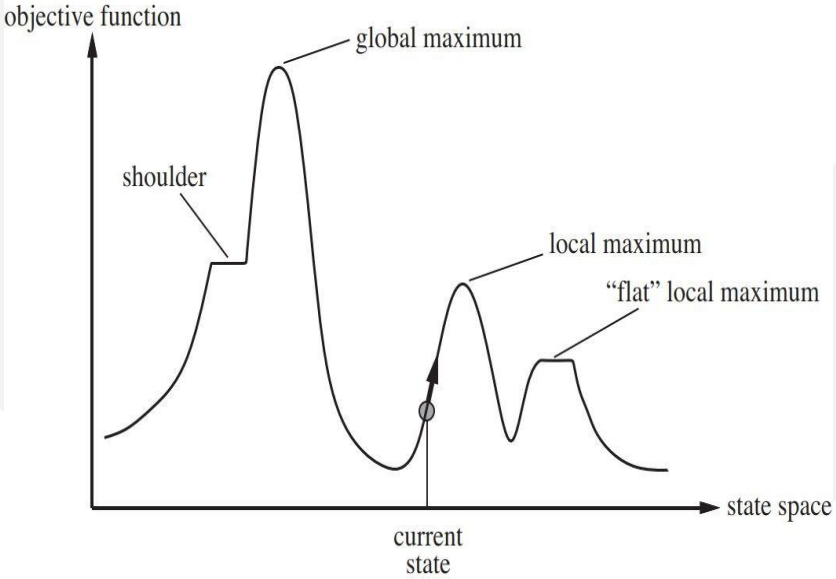
5. Global Maximum

The global maximum is:

- The best possible solution in the entire search space
- No other state has a higher objective value

This is the ideal solution the algorithm aims to find.

Best among all states.



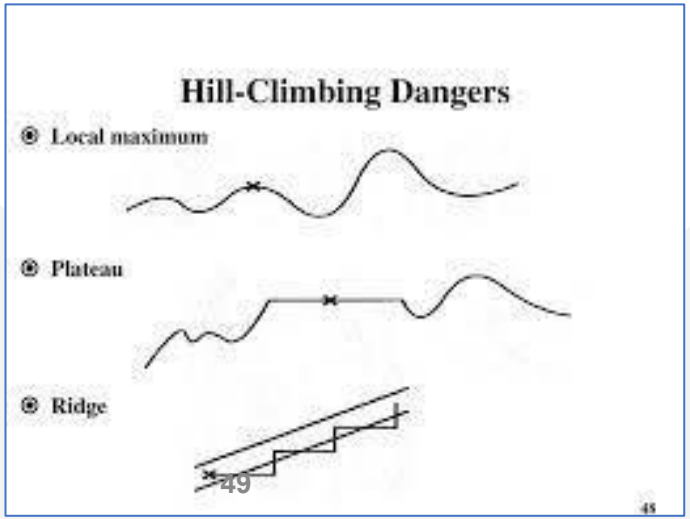
Hill Climbing

- Problem with the Hill Climbing

  - **Local maximum:** At this point, the neighboring states have lower values than the current state. The greedy approach feature will not move the algorithm to a worse off state. This will lead to the hill-climbing process's termination, even though this is not the best possible solution

  - **Plateau:** In this region, the values attained by the neighboring states are the same. This makes it difficult for the algorithm to choose the best direction

  - **Ridge:** The hill-climbing algorithm may terminate itself when it reaches a ridge. This is because the peak of the ridge is followed by downward movement rather than upward movement

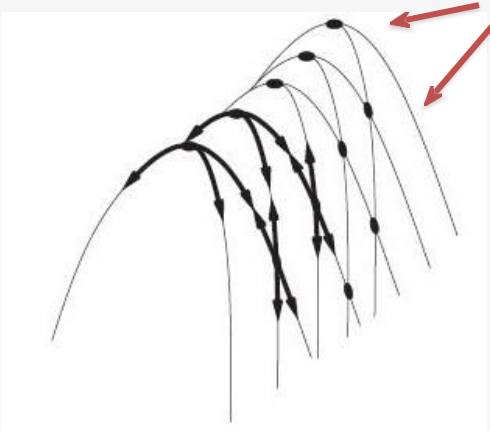


Hill Climbing - Difficulties

Note: these difficulties apply to all local search algorithms, and usually become much worse as the search space becomes higher dimensional

- Ridge problem: every neighbor appears to be downhill
  - But, search space has an uphill (just not in neighbors)

Ridge:
Fold a piece of paper and hold it tilted up at an unfavorable angle to every possible search space step.
Every step leads downhill; but the ridge leads uphill.



States / steps (discrete)

RV College of Engineering®

Implement a Hill-climbing search algorithm to maximize a single variable function f(x).

Hill-Climbing Algorithm

Definition:
Hill-Climbing is a **mathematical optimization algorithm** that starts from an initial solution and moves step-by-step toward a **better neighboring solution**, aiming to reach the **maximum (peak)** of a function.

Used in:
- Function optimization
- Game AI and pathfinding
- Machine Learning hyperparameter tuning
- Feature selection problems

We have:
- An **objective function** f(x) to maximize (or minimize)
- A **current position** x
- **Neighboring solutions** close to x

At each step:
1. Look at nearby points (neighbors).
2. Move to the one with the highest f(x).
3. Stop when no neighbor is better.

Hill-Climbing Algorithm

Step 1: Initialize x_{current} (random or user-given)

Step 2: Compute $f(x_{\text{current}})$

Step 3: Generate neighbors (e.g. $x \pm \Delta$)

Step 4: Evaluate f for each neighbor

Step 5: Move to the best neighbor if it improves f(x)

Step 6: Repeat until no improvement → STOP

Hill-Climbing Algorithm

Inputs	Meaning
$f(x)$	Objective function
x_0	Initial value
Δ	Step size
Max Iterations	Stop limit

Outputs	Meaning
x_{best}	Best value of x found
$f(x_{\text{best}})$	Corresponding maximum output

Example Function

$$f(x) = -(x-3)^2 + 20$$

- Peak (maximum) occurs at **x = 3, f(3) = 20**

x	f(x)
0	11
1	16
2	19
3	20 ✅
4	19
5	16
6	11

Observation:

- Function increases until x=3, then decreases.
- Therefore, **x=3** gives the **global maximum**.

Example Function

Parameters:

$\Delta = 0.5$, initial $x = 0.0$, $f(x) = -(x-3)^2 + 20$

Iter	x_current	Neighbors (x ± 0.5)	f(x_current)	f(Left)	f(Right)	Best f(x)	New x	Move?
0	0.0	-0.5, 0.5	11.00	7.75	13.75	13.75	0.5	Move Right
1	0.5	0.0, 1.0	13.75	11.00	16.00	16.00	1.0	Move Right
2	1.0	0.5, 1.5	16.00	13.75	17.75	17.75	1.5	Move Right
3	1.5	1.0, 2.0	17.75	16.00	19.00	19.00	2.0	Move Right
4	2.0	1.5, 2.5	19.00	17.75	19.75	19.75	2.5	Move Right
5	2.5	2.0, 3.0	19.75	19.00	20.00	20.00	3.0	Move Right
6	3.0	2.5, 3.5	20.00	19.75	19.75	20.00	3.0	Stop

Implementation Steps

The implementation steps given:

1. **Initialization** — Choose initial x, set current solution.
2. **Generate Neighbors** — Small changes to the current x.
3. **Select the Best Neighbor** — Move to the one with higher f(x).
4. **Termination Criteria** — Stop if no improvement or max iterations reached.
5. **Termination Check** — Verify the stop condition.
6. **Repeat or Output** — Repeat until termination, then print x and f(x).

Hill Climbing

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
   inputs: problem, a problem
   local variables: current, a node
                    neighbor, a node

   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
      current ← neighbor
```

RV College of Engineering®

Variants in Hill Climbing

1. Stochastic Hill Climbing

How it works

- From the current state:
  1. Identify **all uphill (better) neighbors**
  2. **Randomly choose one** of them
  3. Move to it
- Selection probability may depend on:
  - **Steepness of improvement**, Bigger improvement → higher probability of reaching the goal.

Key Characteristics

- Does **not always pick the best** uphill move
- Introduces **controlled randomness**
- Explores the search space more **diversely**

2. First-Choice Hill Climbing

- A practical form of **stochastic hill climbing**.
- Successors are:
  - Generated **randomly one by one**
  - The first successor that is **better than the current state** is chosen.
- **Key advantage:**
  - Does **not evaluate all neighbors**
- **Best suited for:**
  - States with **very large number of successors** (thousands or more)
- **Efficiency:**
  - Much faster per step than steepest ascent

Stop searching as soon as you find a better move.

3. Random-Restart Hill Climbing

- Based on the idea:
  "If at first you don't succeed, try again."
- The algorithm:
  - Runs hill climbing **multiple times**
  - Each time starts from a **random initial state**
- **Purpose:**
  - Avoids being trapped in **local maxima**
- **Effectiveness:**
  - Probability of finding the **global maximum increases**
  - Often **complete** in practice for finite spaces

Multiple starts increase the chance of finding the best solution

RV College of Engineering®

Problem with Pure Hill Climbing

- Never accepts downhill (worse) moves
- Gets stuck in **local maxima / minima**
- Cannot escape once no neighbor is better
- **Not complete** (may miss global optimum)

Problem with Pure Random Walk

- Moves randomly, ignores solution quality
- Will eventually find global optimum
- **Extremely inefficient** (very slow)
- Complete (in theory)

Key Idea

Combine:

Efficiency of hill climbing
Completeness of random walk

Simulated annealing balances exploration and exploitation by combining hill climbing and random walk, enabling efficient search while avoiding local optima.

RV College of Engineering®

Annealing

The key idea behind simulated annealing is borrowed from the **physical annealing process** used in metallurgy.

- When a metal is heated to a high temperature, its atoms move freely and explore many configurations.
- As the temperature is slowly reduced, the atoms gradually settle into a **low-energy, stable structure**.
- Similarly, simulated annealing starts with a **high randomness**, allowing many random and even bad moves,
- and then gradually reduces the randomness, so that the search becomes more focused and stable.

At each step, the algorithm selects a **random neighboring state**.

- If this new state is better than the current one, it is always accepted.
- If it is worse, it may still be accepted with a certain **probability**, which depends on two factors:

- How bad the move is, and
- The current temperature.

✔ When the temperature is high, even bad moves have a high chance of being accepted.
✔ As the temperature decreases, the probability of accepting worse moves also decreases.

This gradual reduction in randomness, simulated annealing first **explores** the search space widely and later **exploits** the best regions it has found. If the temperature is lowered slowly enough, simulated annealing is guaranteed (in theory) to reach the **global optimum**.

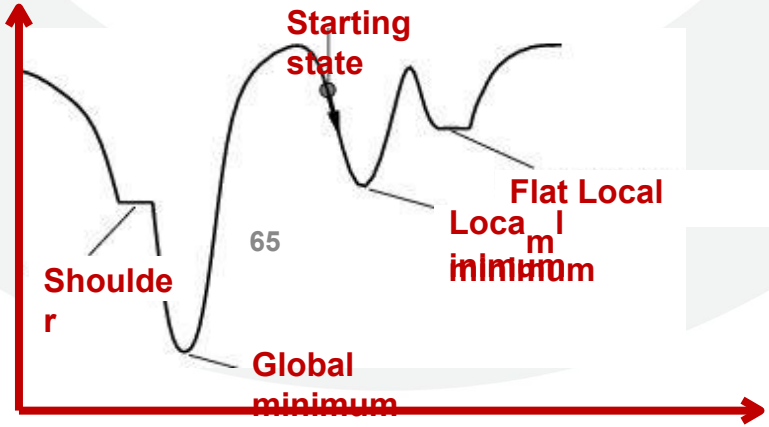
RV College of Engineering®

Simulated Annealing

IDEA: Escape local maxima by allowing some "bad" moves but **gradually decrease** their probability

- The probability is controlled by a parameter called **temperature**
- **Higher temperatures allow more bad moves than lower temperatures**
- **Annealing:** Lowering the temperature gradually **Quenching:** Lowering the temperature rapidly

```
function SIMULATED-ANNEALING( problem, schedule )
    current ← INITIAL-STATE[ problem ]
    for t ← 1 to ∞ do
        T ← schedule[ t ]
        if T = 0 then return current
        next ← a randomly selected successor of current
        ΔE ← VALUE[ next ] – VALUE[ current ]
        if ΔE < 0 then current ← next
        else current ← next with probability e^(−ΔE/T)
```



RV College of Engineering®

Algorithm Explanation

function SIMULATED-ANNEALING(problem, schedule)

This line defines the simulated annealing algorithm. The problem contains information about states, successors, and the evaluation or cost function. The schedule defines how the temperature changes over time.

current ← INITIAL-STATE[problem]

The algorithm begins from an initial state, usually chosen randomly. This is the current solution that will be improved step by step.

for t ← 1 to ∞ do

The algorithm runs iteratively. The variable t represents time or iteration count. In theory, the loop can run forever, but in practice it stops when the temperature becomes very low.

T ← schedule[t]

The temperature at iteration t is obtained from the schedule. The temperature controls how much randomness is allowed in the search. At the beginning, the temperature is high; later, it becomes low.

if T = 0 then return current

When the temperature reaches zero, no worse moves should be accepted. At this point, the algorithm terminates and returns the current state as the final solution.

RV College of Engineering®

Algorithm Explanation

next ← a randomly selected successor of current

A neighboring state of the current state is selected randomly. Unlike hill climbing, the algorithm does not look at all neighbors or choose the best one. Random selection helps the algorithm explore different regions of the search space.

ΔE ← VALUE[next] − VALUE[current]

This line computes the change in value (or energy). If ΔE is positive, the new state is better than the current one. If ΔE is negative, the new state is worse.

if ΔE > 0 then current ← next

If the new state has a higher value (or lower cost), it is always accepted. This behavior is similar to hill climbing and ensures progress toward better solutions.

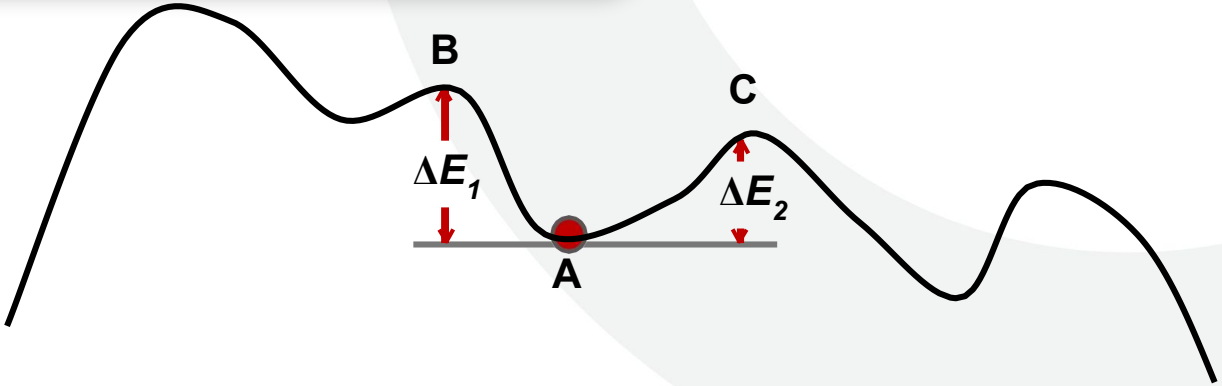
else current ← next with probability e^(ΔE / T)

If the new state is worse, it may still be accepted with a certain probability. This probability depends on how bad the move is (ΔE) and the current temperature (T). When the temperature is high, even bad moves are likely to be accepted. When the temperature is low, bad moves are rarely accepted.

RV College of Engineering®

How Simulated Annealing Works ?

Probability of making a bad move = $e^{-\Delta E/T} = \frac{1}{e^{\Delta E/T}}$



Since □E_1 > □E_2 moving from A to C is exponentially more probable than moving from A to B

Properties-Simulated Annealing

- If the temperature **T is decreased slowly enough**, simulated annealing is **guaranteed to reach the global optimum** with probability approaching 1.
- Such slow cooling requires **very long runtime**, which is impractical for real-world problems.
- In practice, a **faster temperature schedule** is used to fit the available **time and computational budget**, resulting in **near-optimal solutions**.
- Despite this, simulated annealing **performs very well in practice** and effectively escapes local optima.
- It is **widely used** in large-scale optimization problems such as **VLSI layout**, **airline scheduling**, and **timetabling**.

Local-beam search

- Local beam search is an extension of hill climbing that addresses the limitation of keeping only one state in memory.
- Instead of tracking a single current state, the algorithm keeps track of k states simultaneously.
- The algorithm begins by generating k random initial states. At each step, it generates all successors of all k states. If any successor is a goal state, the search stops immediately. Otherwise, from the complete set of generated successors, the algorithm selects the k best states according to the evaluation function and repeats the process.
- By keeping multiple states, local beam search increases the chance of finding a good region of the search space while still using relatively little memory compared to systematic search methods.

Local Beam Search vs Random-Restart Hill Climbing

At first glance, local beam search may appear similar to running k random-restart hill-climbing searches in parallel. However, the two are fundamentally different.

- In random-restart hill climbing, each search runs independently, without sharing any information. In contrast, local beam search shares information across all k states.
- States that generate strong successors effectively guide the others toward more promising regions of the search space. Poor search paths are quickly abandoned, and computational effort is focused where the most progress is being made.
- This cooperation between states makes local beam search more directed and efficient than random restarts.

RV College of Engineering®

A major drawback of local beam search is loss of diversity. Over time, the k states may become very similar and cluster in the same region of the search space. When this happens, the algorithm behaves like a slower version of hill climbing, offering little advantage despite using k states.

Imagine **k students** searching for treasure on a large field.
At the start, all students begin at **different places**.
After one round, the students who find slightly better clues are copied, and the weaker ones are dropped.
Very soon, **all students move to the same area** because:

- The algorithm always keeps only the **best k states**
- These best states often come from the **same region**

Now:

- Everyone is exploring **the same small area**
- Other promising areas are **completely ignored**

So even though we have **k search paths**, they all behave like:
This is called **loss of diversity**.

Stochastic Beam Search :

Instead of keeping only the top k students, we:

- Prefer better students
- But sometimes keep weaker ones too

This way:

- Some students stay in different regions
- Search remains diverse
- More chances of finding a better hill later

This is stochastic beam search.

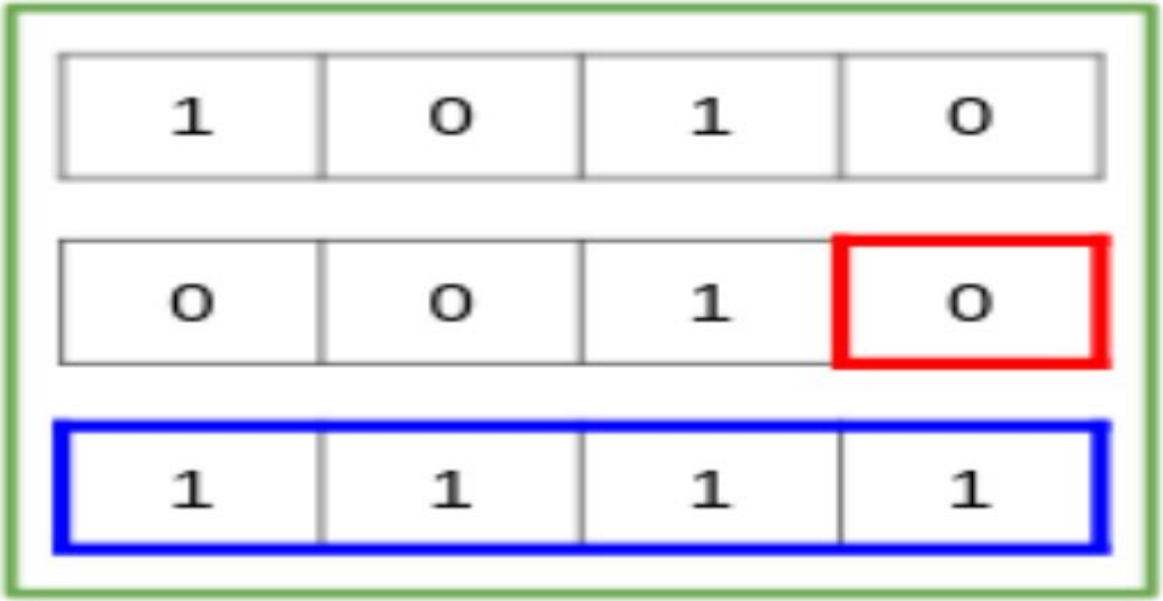
RV College of Engineering®

Genetic Algorithms

Genetic Algorithms (GAs) are **search and optimization algorithms inspired by natural evolution**.

In a genetic algorithm, each **state (solution)** is represented as a **string of symbols**, called a **chromosome**.
Each symbol in the string is called a **gene**, and it takes values from a **finite alphabet** (for example, {0,1} or letters or numbers).

The algorithm begins with a **population of k randomly generated states**.
Instead of improving one solution at a time (like hill climbing), genetic algorithms work on **many solutions simultaneously**.
Each solution is evaluated using a **fitness function**, which measures how good that solution is. Solutions with **higher fitness** are more likely to be selected to produce the next generation.



Population is the all possible results of the given problem (in figure, whole green border portion). **Chromosome** (sometimes, it is also called individual) is a one solution from the population set (blue border portion). **Gene** is the single element position of the chromosome (red). Fitness represents the improvement of generation by generation.

RV College of Engineering®

Genetic Algorithms

New solutions (offspring) are created using two main genetic operators:

- Crossover combines parts of two fit parent solutions to produce new offspring. This allows good features from different parents to be passed on and combined.
- Mutation randomly changes a small part of a solution with a very low probability. This introduces new genetic material into the population and prevents the search from getting stuck too early.

Over many generations, the population evolves, and solutions gradually become better and better.

Simple Analogy

- Population → group of students
- Fitness → exam score
- Crossover → mixing answers from two toppers
- Mutation → small random change in an answer
- Goal → best possible solution

RV College of Engineering®

Genetic Algorithm:

https://www.linkedin.com/pulse/genetic-algorithm-shreya-thakrar/

There are basically five steps of Genetic Algorithm:

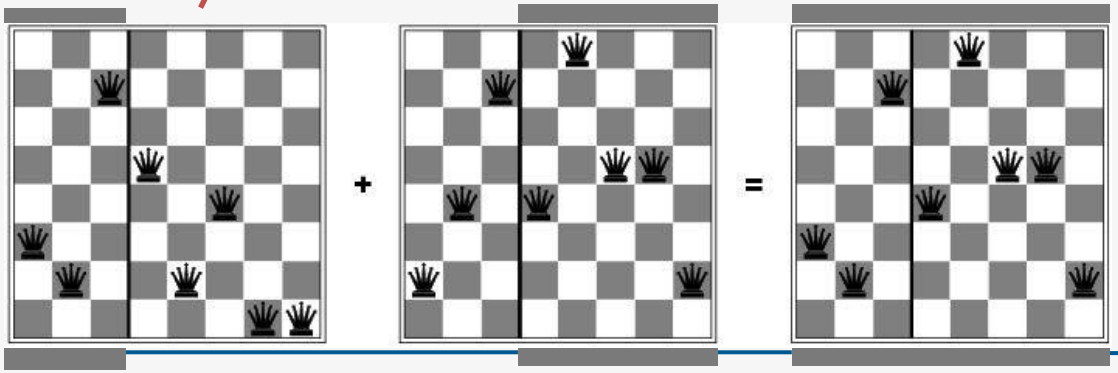
- Initial population
- Fitness function evaluation
- Selection
- Crossover
- Mutation

Genetic Algorithms – 8 Queens Problem

®

Go, change the world

Fitness function: # non-attacking pairs (min 0, max = 8×7 / 2 = 28)



Population fitness = 24+23+20+11 = 78

P(Gene-1 is chosen)
= Fitness of Gene-1 / Population fitness
= 24 / 78 = 31%

P(Gene-2 is chosen)
= Fitness of Gene-2 / Population

RV College of Engineering®

Chromosome Representation

- Each solution is written as an **8-digit string**
- Each digit represents the **row position of a queen** in each column

  Example: 24748552
- Column number is fixed (1 to 8), value tells the row

  So every string = **one complete board configuration**

Fitness Function

- Fitness = **number of non-attacking queen pairs**
- Maximum possible non-attacking pairs = $\frac{8 \times 7}{2} = 28$

  Higher fitness → better solution

  Example from slide:
- Gene-1 fitness = 24
- Gene-2 fitness = 23
- Gene-3 fitness = 20
- Gene-4 fitness = 11

Population Fitness

Add all fitness values:

24+23+20+11=78

This total is used for probability-based selection.

Selection (Roulette Wheel Selection)

- Probability of selecting a chromosome = $\frac{\text{Fitness of chromosome}}{\text{Total population fitness}}$

Example:

- Probability(Gene-1) = 24 / 78 ≈ 31%
- Probability(Gene-2) = 23 / 78 ≈ 29%

Better chromosomes have higher chance, but weaker ones still have some chance.

Crossover

- Two selected parents exchange parts of their strings
- Produces a **new offspring**
- Offspring inherits good traits from both parents

This helps **combine good partial solutions**.

Overall Meaning

- Population improves **generation by generation**
- Poor solutions die out
- Good traits spread
- Eventually reaches a **valid 8-Queens solution**

Genetic Algorithms

```
function GENETIC-ALGORITHM(population, FITNESS-FN) returns an individual
  inputs: population, a set of individuals
          FITNESS-FN, a function that measures the fitness of an individual

  repeat
     new_population ← empty set
     for i = 1 to SIZE(population) do
        x ← RANDOM-SELECTION(population, FITNESS-FN)
        y ← RANDOM-SELECTION(population, FITNESS-FN)
        child ← REPRODUCE(x, y)
        if (small random probability) then child ← MUTATE(child)
        add child to new_population
     population ← new_population
  until some individual is fit enough, or enough time has elapsed
  return the best individual in population, according to FITNESS-FN
```

```
function REPRODUCE(x, y) returns an individual
  inputs: x, y, parent individuals

  n ← LENGTH(x); c ← random number from 1 to n
  return APPEND(SUBSTRING(x, 1, c), SUBSTRING(y, c + 1, n))
```

Game Playing

Game Playing

Introduction

- A Game playing is one of the oldest areas of endeavor in artificial intelligence
- In 1950, almost as soon as computers became programmable, the first chess programs were written by Claude Shannon (the inventor of information theory) and by Alan Turing.
- The presence of an opponent makes the decision problem somewhat more complicated than the search problems discussed.
- The opponent introduces uncertainty, because one never knows what he or she is going to do. In essence, all game-playing programs must deal with the **Contingency Problem**

Game Playing

Game as a Search Problem

- A game can be defined as a type of search in AI, which can be formalized of the following elements:
  - **Initial state:** It specifies how the game is set up at the start
  - **Player(s):** It specifies which player has moved in the state space
  - **Action(s):** It returns the set of legal moves in state space
  - **Result(s, a):** It is the transition model, which specifies the result of moves in the state space.
  - **Terminal-Test(s):** Terminal test is true if the game is over, else it is false at any case. The state where the game ends is called terminal states.
  - **Utility(s, p):** A utility function gives the final numeric value for a game that ends in terminal states s for player p. It is also called payoff function. For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, ½. And for tic-tac-toe, utility values are +1, -1, and 0

Game Playing (Contd.)

Perfect Decisions in Two-Persons Game

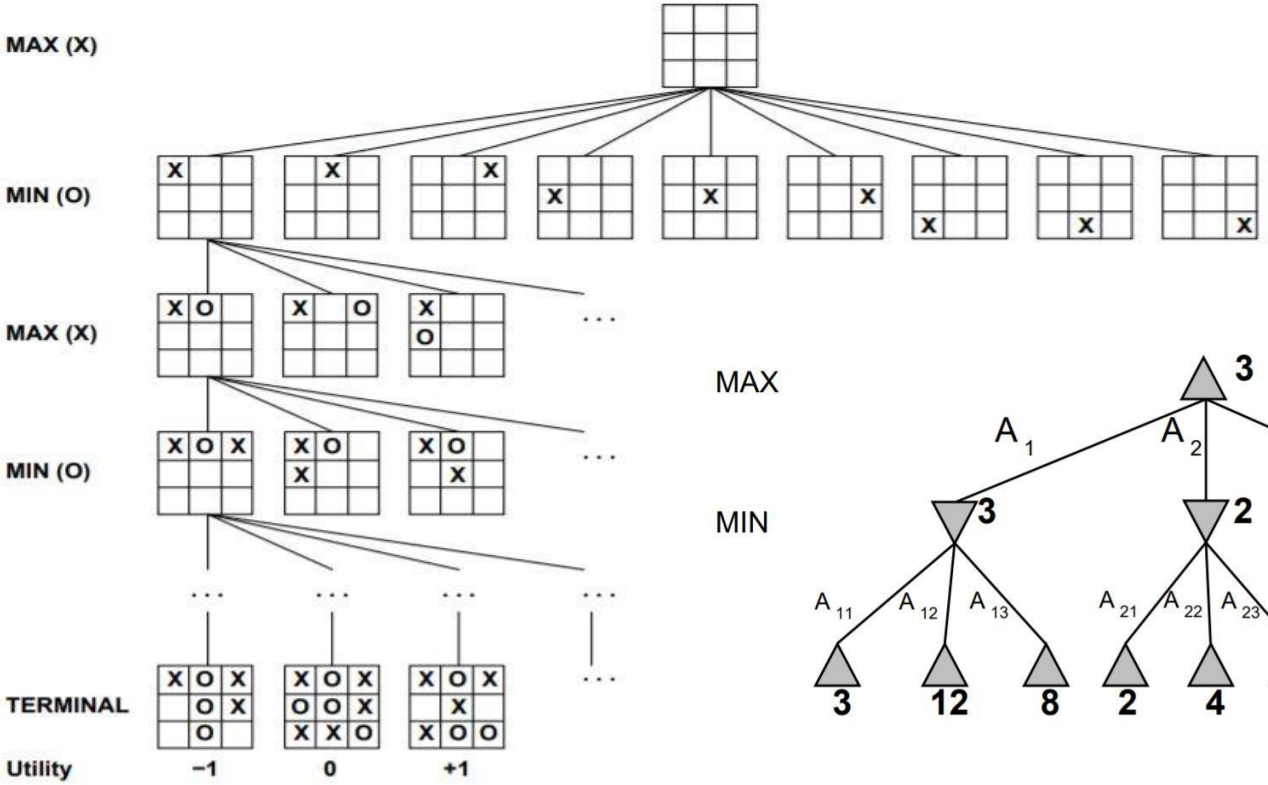
- Consider the general case of a game with two players, whom we will call MAX and MIN
- MAX moves first, and then they take turns moving until the game is over
- At the end of the game, points are awarded to the winning player
- A game can be formally defined as a kind of search problem with the following components
  - The initial state, which includes the board position and an indication of whose move it is.
  - A set of operators, which define the legal moves that a player can make
  - A terminal test, which determines when the game is over. States where the game has ended are called terminal states
  - A utility function (also called a payoff function), which gives a numeric value for the outcome of a game. In chess, the outcome is a win, loss, or draw, which we can represent by the values +1, —1, or 0.

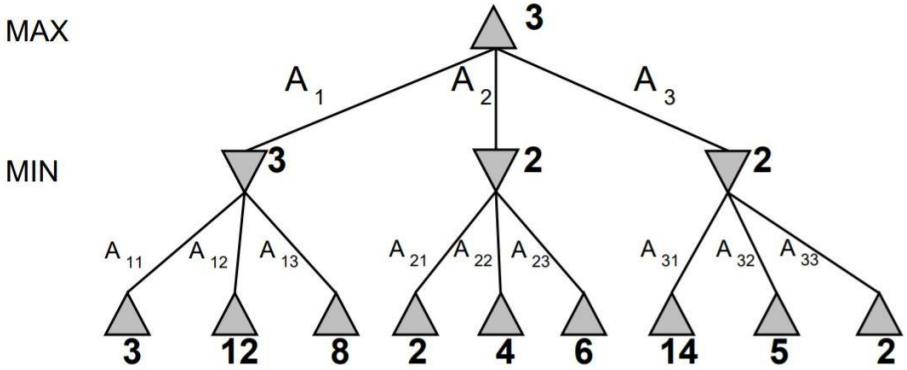
Game Playing (Contd.)

Perfect Decisions in Two-Persons Game

- If this were a normal search problem, then all MAX would have to do is search for a sequence of moves that leads to a terminal state that is a winner
- From the initial state, MAX has a choice of nine possible moves
- Play alternates between MAX placing x's and MIN placing o's until we reach leaf nodes corresponding to terminal states: states where one player has three in a row or all the squares are filled.
- The number on each leaf node indicates the utility value of the terminal state from the point of view of MAX; high values are assumed to be good for MAX and bad for MIN (which is how the players get their names).
- It is MAX'S job to use the search tree (particularly the utility of terminal states) to determine the best move
- Even a simple game like Tic-Tac-Toe is too complex to show the whole search tree

RV College of Engineering®





Game Playing (Contd.)

Minimax Algorithm

- The minimax algorithm is designed to determine the optimal strategy for MAX, and thus to decide what the best first move is
- The algorithm consists of five steps:
  - Generate the whole game tree, all the way down to the terminal states
  - Apply the utility function to each terminal state to get its value
  - Use the utility of the terminal states to determine the utility of the nodes one level higher up in the search tree
  - Continue backing up the values from the leaf nodes toward the root, one layer at a time
  - Eventually, the backed-up values reach the top of the tree; at that point, MAX chooses the move that leads to the highest value

Game Playing (Contd.)

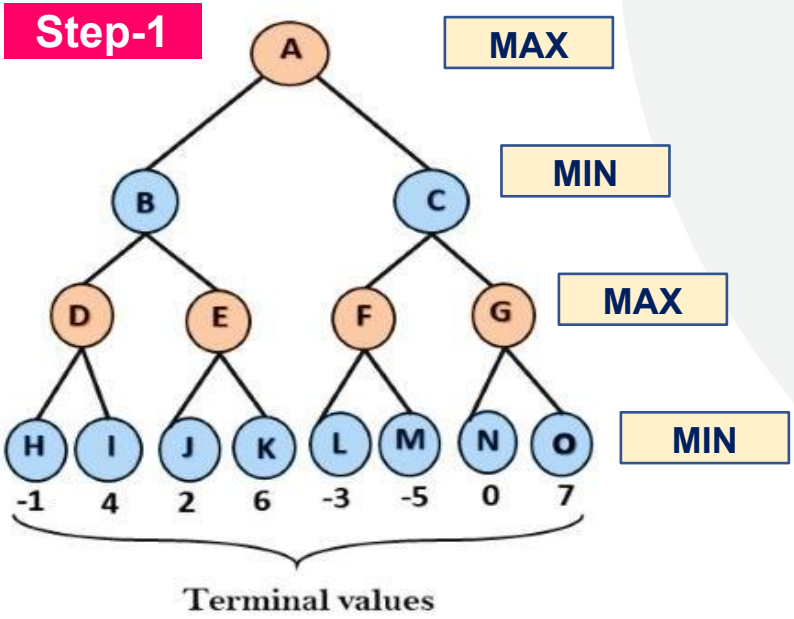
Minimax Algorithm

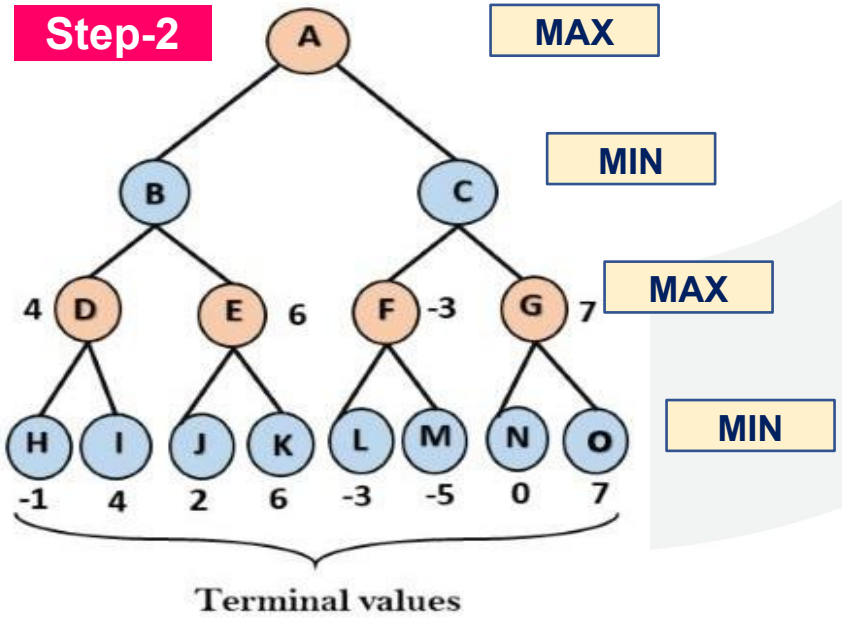
- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory
- It provides an optimal move for the player assuming that opponent is also playing optimally
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game
- In this algorithm two players play the game, one is called MAX and other is called MIN
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value
- The minimax algorithm performsa depth-first search algorithm for the exploration of the complete game tree
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion

Game Playing (Contd.)

RV College of Engineering®

Minimax Algorithm

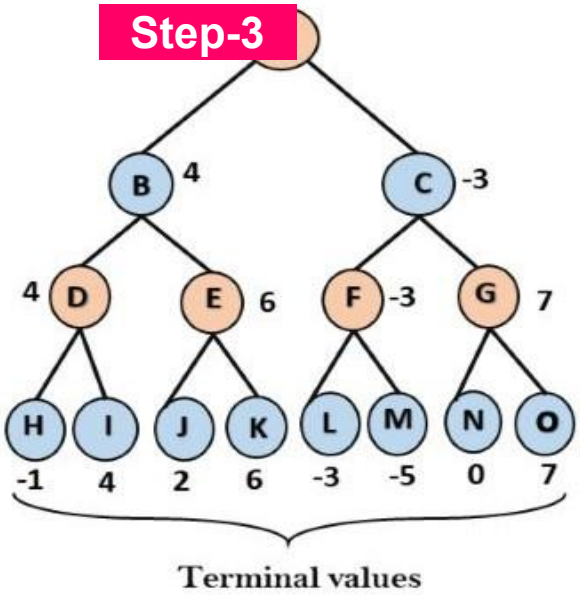


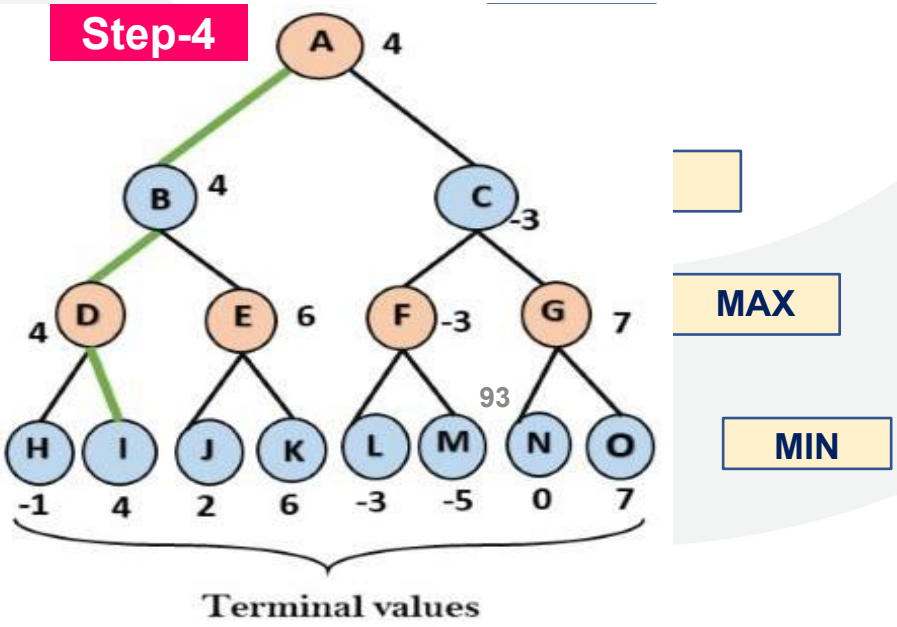


Game Playing (Contd.)

RV College of Engineering®

Minimax Algorithm





→ Minimax Algorithm Pseudocode

```
function MINIMAX-DECISION(game) returns an operator

  for each op in OPERATORS[game] do
      VALUE[op] ← MINIMAX-VALUE(APPLY(op, game), game)
  end
  return the op with the highest VALUE[op]
```

```
function MINIMAX-VALUE(state, game) returns a utility value

  if TERMINAL-TEST[game](state) then
      return UTILITY[game](state)
  else if MAX is to move in state then
      return the highest MINIMAX-VALUE of SUCCESSORS(state)
  else
      return the lowest MINIMAX-VALUE of SUCCESSORS(state)
```

Game Playing (Contd.)

Imperfect Decisions

- The minimax algorithm assumes that the program has time to search all the way to terminal states, which is usually not practical
- The program should cut off the search earlier and apply a heuristic evaluation function to the leaves of the tree
- In other words, the suggestion is to alter minimax in two ways
  - The utility function is replaced by an evaluation function EVAL, and the terminal test is replaced by a cutoff test CUTOFF-TEST

Game Playing (Contd.)

Alpha-Beta Pruning

- In minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree.
- Since we cannot eliminate the exponent, but we can cut it to half
- Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called pruning.
- This involves two threshold parameter Alpha and beta for future expansion, so it is called alpha-beta pruning. It is also called as Alpha-Beta Algorithm
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree

Game Playing (Contd.)

Alpha-Beta Pruning

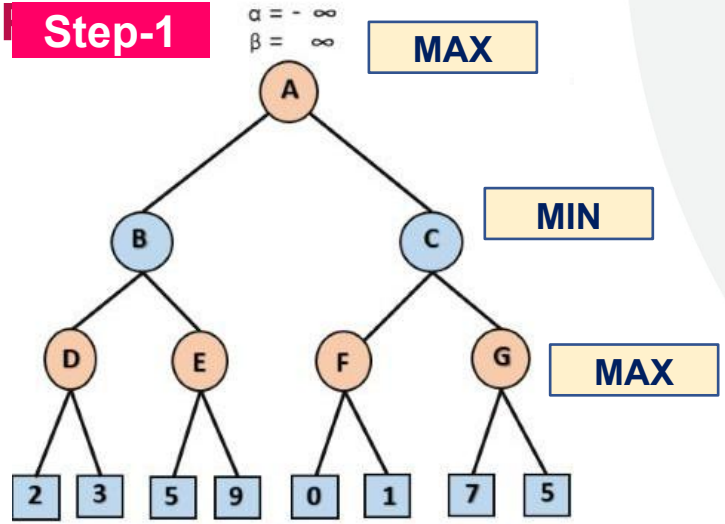
- The two-parameter can be defined as:
  - Alpha: The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is -∞.
  - Beta: The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is +∞.
- The **Alpha-beta** pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow
- Hence by pruning these nodes, it makes the algorithm fast
- Points to remember
  - The Max player will only update the value of alpha
  - The Min player will only update the value of beta
  - While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta
  - We will only pass the alpha, beta values to the child nodes

Game Playing (Contd.)

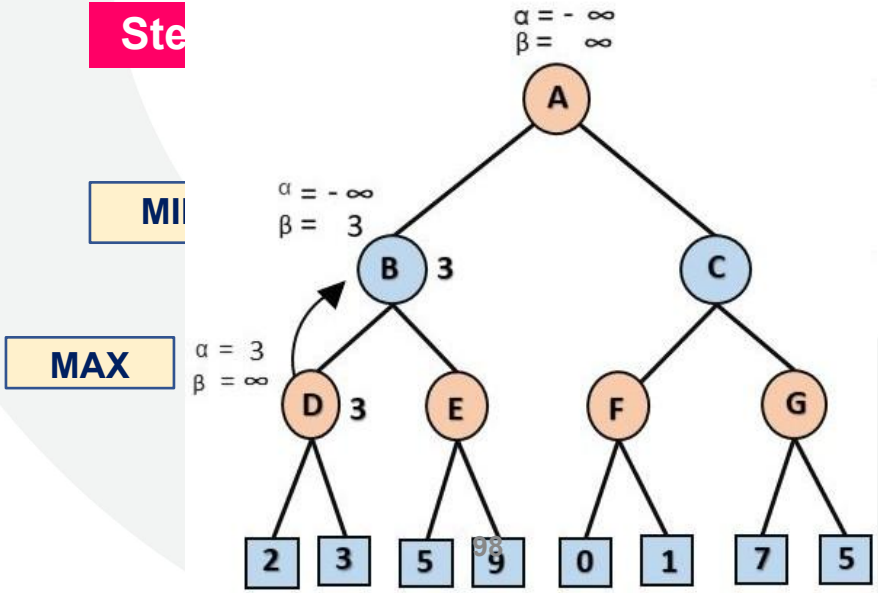
RV College of Engineering®

Alpha-Beta

Step-1



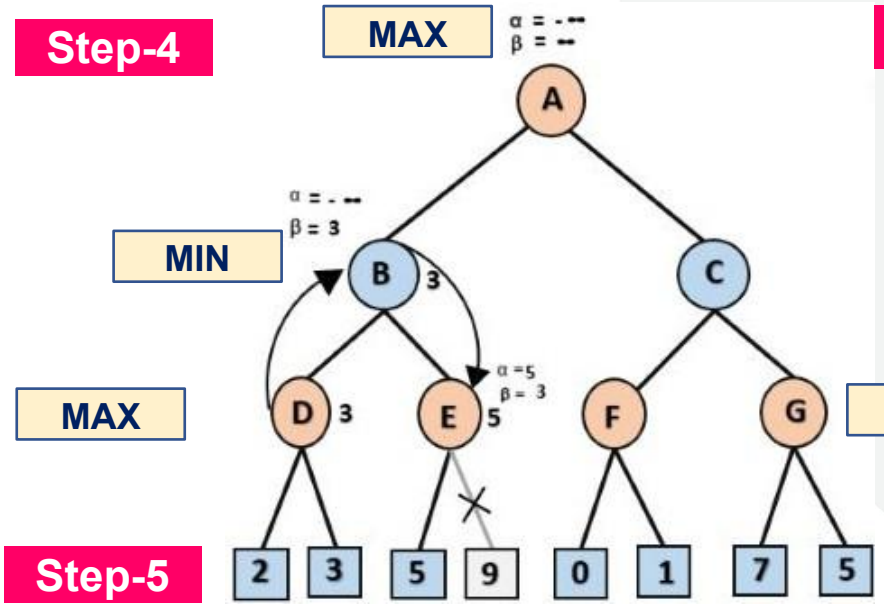
Step-2

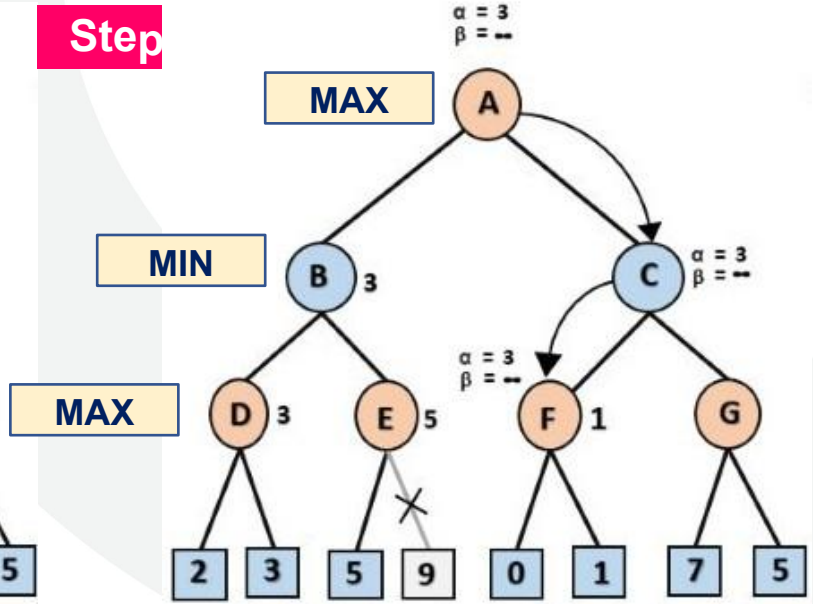


At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3

Game Playing (Contd.)

Alpha-Beta Pruning

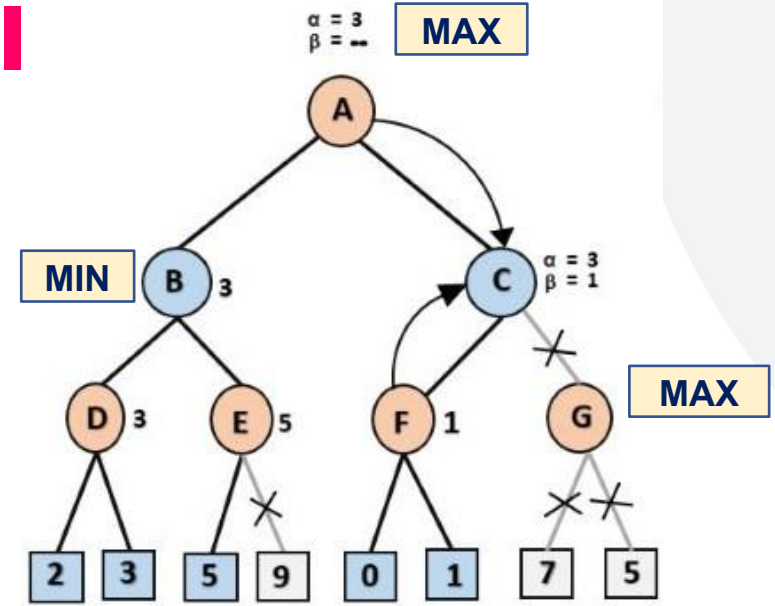


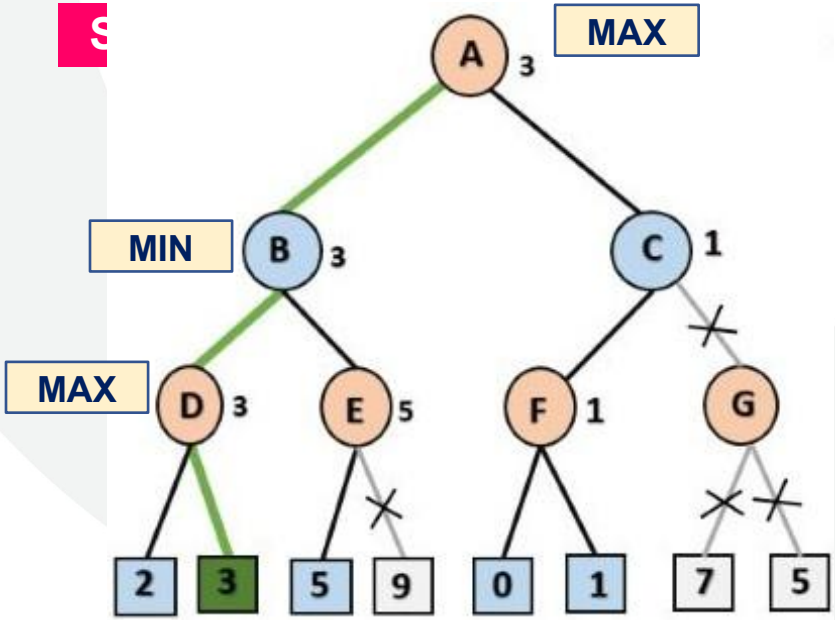


At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C

Game Playing (Contd.)

Alpha-Beta Pruning





Alpha Beta Pruning

RV College of Engineering®

- **function minimax(node, depth, alpha, beta, maximizingPlayer)**

•node: the current position or state in the game tree.
•depth: how deep we are in the tree (used to stop recursion).
•alpha: best value that the **maximizing** player can guarantee so far.
•beta: best value that the **minimizing** player can guarantee so far.
•maximizingPlayer: True if it's MAX's turn, False if it's MIN's turn.
Think of alpha as the **lower bound** for MAX (minimum score MAX will accept) and beta as the **upper bound** for MIN (maximum score MIN will allow).

- **Termination Cases**
- **if depth == 0 or node is a terminal node:**
- **return static evaluation of node**

●**Base case** of recursion.
•If you've reached a leaf node or the maximum search depth, you simply **evaluate** the board (for example, using a heuristic function).
•This returns a numeric score representing how good this node is for MAX.

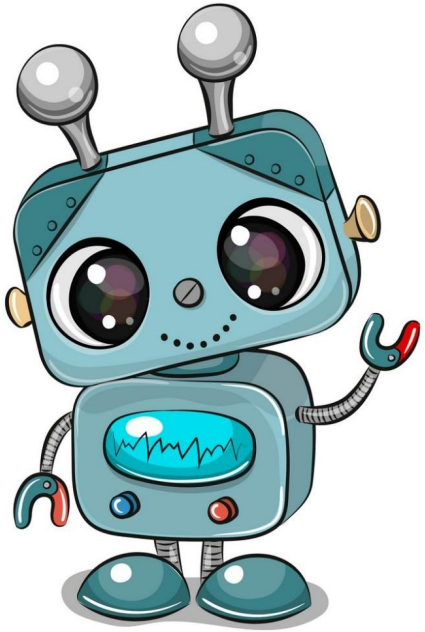
```
if maximizingPlayer:
    maxEva = -infinity
    for each child of node:
        eva = minimax(child, depth-1, alpha, beta, False)
        maxEva = max(maxEva, eva)
        alpha = max(alpha, maxEva)
        if beta <= alpha:
            break
    return maxEva
```

1. Initialize maxEva = -∞ (worst possible value).
2. Loop through each child (possible move).
3. Recursively call minimax for the opponent (MIN player).
4. Update the best value found so far.
5. Update alpha:
   - α = max(α, current best)
6. Prune when β < α:

```
if not maximizingPlayer:
    minEva = +infinity
    for each child of node:
        eva = minimax(child, depth-1, alpha, beta, True)
        minEva = min(minEva, eva)
        beta = min(beta, minEva)
        if beta <= alpha:
            break
    return minEva
```

1. Initialize minEva = +∞ (worst for MIN).
2. For each child:
   - Recursively evaluate assuming MAX moves next.
3. Update with the smaller of the current value or the child value.
4. Update β = min(β, current best) (MIN tries to lower the bound).
5. Prune when β ≤ α (MAX already found something better elsewhere).
6. Return minEva as MIN's best move value.

RV College of Engineering®



Thank You